Credit Risk Analysis: Retail Mortgage Loans

Logistic Regression Probability of Default Model

Antonio Melacini

December 2025

Overview

We create a 1-month-ahead probability of default (PD) model using logistic regression with panel data comprised of loan origination characteristics, monthly loan performance features and macroeconomic covariates.

We have natural right-censoring present in the data as well as censoring that we induce to enforece our simplified definition of the event of default. Due to the censoring, we can also view the model as a hazard rate model.

We form a sample by combining data from 2013, 2016 and 2020 vintages sourced from the Freddie Mac Single Family Loan-Level Dataset (SFFLD). These origination years are chosen to provide a sample of loans originating in a variety of macroeconomic environments and to help identify the effects of age of loan separately from calendar time.

We do a first pass of variable selection by including economically justified covariates and inspecting the empricial relationships between our covariates and the logistic transform of the hazard. We then see if we can drop any unecessary covariates using Lasso. After variable selection, we train the model on the training set and evaluate the performance on a held-out test set.

One can skip to the "Interpretation" or "Prediction Under Stress Testing Scenarios" sections near the end of this document to see the final results where we interpret selected model parameters and use the model to predict default rates across different economic stress scenarios.

Model Definition

We define the event of default as 90+ days past due (or 3+ missed monthly payments). Some borrowers never make another monthly payment after "default", although there are also many borrowers that may make continue to make some future payments and then possibly "default" again. This definition is a simplifying assumption that allows us to have more cases of "default" than if we only included strict defaults. We wish to model default as an absorbing state, meaning that a lender remains in default once they default. To enforce this, for each loan we drop the rows for months following the month of default. More details on the censoring that we encoded in the data is discussed after introducing the model.

We will fit the following logistic regression model (Binomial GLM with canonical link):

$$\begin{aligned} Y_{it} &\sim \text{Bernoulli}(\pi_{it}) \quad \text{independently} \\ \pi_{it} &= \text{PD}_{it}^{\text{1mo}} \\ \text{logit}(\pi_{it}) &= x_{it}^T \beta^{(L)} + z_t^T \beta^{(M)} \end{aligned}$$

where:

- $\text{logit}(\pi) = \ln\Big(\pi/(1-\pi)\Big)$ is the logistic function,
- $Y_{it} = \mathbb{1}[\text{loan } i \text{ defaults in month } t+1]$,
- $\pi_{it} = \mathbb{P}(Y_{it} = 1) = \text{PD}_{it}^{\text{1mo}}$ is the probability of default in month $t+1$ (given survival up to time t),
- x_{it} is a vector of covariates for loan i at month t,
- $\beta^{(L)}$ is a vector of coefficients (parameters) for the loan-specific associations,
- z_t is a vector of exogenous macroeconomic covariates at month t (these are observed before t), and
- $\beta^{(M)}$ is a vector of coefficients for the macroeconomic associations.

To be clear about independence, we are still opearting on the classical GLM assumption that the random outcomes are conditionally independent given the covariates.

This can also be viewed as a discrete hazard model:

Let T_i be the non-negative integer-valued survival time (time of default) for loan i. The hazard (instantaneous risk) function for loan i at month $t+1$ is

$$\begin{aligned} h_i(t+1) &= \mathbb{P}(T_i = t+1 | T_i \geq t+1) \\ &= \mathbb{P}(\text{default in month } t+1 | \text{no default as of month } t) \\ &= \mathbb{P}(Y_{it} = 1) \\ &= \pi_{it} \end{aligned}$$

This interpretation is valid because we removed rows for loans in the months after they defaulted[1] and we dropped the final month of censored observations[2], so each loan-month observation represents a period during which the loan is *at risk* of default.

note: next step is clean up EDA section, then can mostly just run code from earlier to fit models and optimize with CV etc., then can do final interpretation + predictions

```
# Load libraries
library(tidyverse) # SQL-like data manipulation and more (e.g. dplyr)
```

```
## Warning: package 'stringr' was built under R version 4.4.2
```

[1] Since "default" was defined as 90+ DPD, this means we removed observations in months $t+1, t+2, \ldots$ after $Y_{it} = 1$ even if there may be some loans which ended up returning to a non-default state.

[2] At the final month in our dataset, whether or not a loan defaults next month is unknown.

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

```
library(ggplot2) # highly customizable visualizations
library(fredr) # FRED API
```

```
## Warning: package 'fredr' was built under R version 4.4.3
```

```
library(corrplot) # correlation matrix heat maps
```

```
## Warning: package 'corrplot' was built under R version 4.4.3
```

```
## corrplot 0.95 loaded
```

```
library(performance) # VIF calculations and visualizations
```

```
## Warning: package 'performance' was built under R version 4.4.3
```

```
library(knitr) # polished display of tables
```

Load and Clean Loan Data

```
# Import data
orig_df <- read.csv("C:/Users/acubi/OneDrive/Desktop/Projects-Code-Certificates/Credit-Risk-Mo
perf_df <- read.csv("C:/Users/acubi/OneDrive/Desktop/Projects-Code-Certificates/Credit-Risk-Mo
```

Note: we're using the definition of the "state" variable from the notebook "Mortgage-Credit-Risk-DTMC.ipynb".

We are concerned with modeling the event of default in the *next* month using information available in the *current* month.

```
# Create indicator variable for event of default next month and
#   drop loans from data (remove from at-risk set) in months after defaulting
perf_df <- perf_df %>%
  arrange(loan_sequence_number, ymd(monthly_reporting_period)) %>%
  group_by(loan_sequence_number) %>%
  mutate(
    exit_now = (state %in% c("Default","Prepaid")),
    keep = (cumsum(exit_now) == 0), # keep rows for a loan strictly before default
    next_state = lead(state),
    y = as.integer(next_state == "Default") # response variable
  ) %>%
```

```
  ungroup() %>%
  filter(keep, !is.na(y)) %>%
  select(-exit_now, -keep)

# Merge origination and monthly data
full_df <- perf_df %>% left_join(orig_df, by = "loan_sequence_number")

# Delete perf_df to save space
rm(perf_df)

# Note: for any fixed loan, the origination variables are constant across months in full_df


# Drop right-censored rows (last time point for each loan)
full_df <- full_df %>% filter(!is.na(y))

# Sort on (loan ID, time)
full_df <- full_df %>% arrange(loan_sequence_number, ymd(monthly_reporting_period))

# Drop loans with NA values for LTV ratio at origination
na_ltv_ids <- full_df %>%
  filter(orig_ltv == 999) %>%
  pull(loan_sequence_number) %>% unique()
na_ltv_ids %>% length()
```

```
## [1] 7
```

```
full_df %>% filter(loan_sequence_number %in% na_ltv_ids, y == 1) %>%
  count() # check that dropping this wouldn't get rid of many default events (it doesn't)
```

```
## # A tibble: 1 x 1
##       n
##   <int>
## 1     1
```

```
full_df <- full_df %>% filter(!(loan_sequence_number %in% na_ltv_ids))

# Check out loans with NA values for DTI ratio at origination
na_dti_ids <- full_df %>%
  filter(orig_dti == 999) %>%
  pull(loan_sequence_number) %>% unique()
na_dti_ids %>% length()
```

```
## [1] 33085
```

```
full_df %>% filter(loan_sequence_number %in% na_dti_ids, y == 1) %>%
  count() # check that dropping this wouldn't get rid of many default events (it does!)
```

```
## # A tibble: 1 x 1
##       n
```

```
##   <int>
## 1  1708
```

```
# Create indicator for missing DTI values and impute the missing ones with median
clean_dti_vec <- orig_df$orig_dti[orig_df$orig_dti != 999]
med_dti <- median(clean_dti_vec)
full_df <- full_df %>%
  mutate(dti_missing = (orig_dti == 999),
         orig_dti_imp = ifelse(test=(orig_dti==999),
                               yes=med_dti, no=orig_dti))


# Drop loans with NA values for credit score
na_cs_ids <- full_df %>%
  filter(credit_score == 9999) %>%
  pull(loan_sequence_number) %>% unique()
na_cs_ids %>% length()
```

```
## [1] 27
```

```
full_df %>% filter(loan_sequence_number %in% na_cs_ids, y == 1) %>%
  count() # check that dropping this wouldn't get rid of many default events (it doesn't)
```

```
## # A tibble: 1 x 1
##       n
##   <int>
## 1     0
```

```
full_df <- full_df %>% filter(!(loan_sequence_number %in% na_cs_ids))

# Normalize loan age into percentage of loan term
full_df$loan_age_pct <- full_df$loan_age / full_df$orig_loan_term

# Create variable for proportion of original unpaid principal left unpaid
full_df$upb_pct <- full_df$current_actual_upb / full_df$orig_upb

# Create region variable (to use instead of state)
extra_abbs <- c("DC", "PR", "VI", "GU", "AS", "MP")
extra_regs <- c("South", "South", "South", "West", "West", "West")
custom_abb <- c(state.abb, extra_abbs)
custom_region <- c(as.character(state.region), extra_regs)
full_df <- full_df %>%
  mutate(region = custom_region[match(property_state, custom_abb)])

# Check levels of PPM flag and amortization type
orig_df$amortization_type %>%
  as.factor() %>% levels()
```

```
## [1] "FRM"
```

```r
orig_df$ppm_flag %>%
  as.factor() %>% levels()
```

```
## [1] "N"
```

```r
# Ensure orig_df only keeps IDs that we use for the full panel data
orig_df <- orig_df %>%
  filter(loan_sequence_number %in% unique(full_df$loan_sequence_number))


# Select relevant variables for modeling
full_df <- full_df %>% select(
  loan_sequence_number, # ID
  monthly_reporting_period, # time
  y, # response (indicator for default next month)

  # monthly performance features (time-varying)
  deliq_num, # number of missed payments
  loan_age, # age of loan in months
  loan_age_pct, # age of loan as percentage of loan term
  upb_pct, # percentage of original unpaid principal left unpaid

  # origination features (constant over time for fixed loan)
  orig_upb, # unpaid balance at origination (original loan amount)
  quarter, # quarter of year when loan was originated
  credit_score, # FICO credit score
  orig_ltv, # loan-to-value ratio at origination
  orig_dti_imp, # debt-to-income ratio at origination (imputed with median)
  dti_missing, # indicator for DTI at origination being missing
  orig_interest_rate, # interest rate on loan at origination
  orig_loan_term, # length of the loan in months
  occupancy_status, # e.g. primary residence, investment property, etc.
  region, # region of USA (north/east/south/west) where property is located
  loan_purpose, # e.g. Cash-Out Refinance mortgage
) %>% rename(
  date = monthly_reporting_period,
  orig_rate = orig_interest_rate,
  orig_qtr = quarter
)
```

The official document with the full data dictionary and user guide for SFLLD can be found at user guide.

There exist variables which may be useful that we did not use, such as "Estimated LTV" in the monthly performance data which relies on Freddie Mac's proprietary Automated Valuation Model. These variables are omitted so that all variables in the final model are those which are readily available at a typical bank or credit union.

We drop loans with NA values for LTV at origination since it is an extremely small portion of our

sample and none of those loans end up defaulting.

For loans with NA values for DTI at origination (`orig_dti`), there are actually a material amount which end up defaulting, so we can't simply drop these loans. We create an indicator variable called `dti_missing` for `orig_dti` having an NA value, and we create a new variable `orig_dti_imp` which is equal to `orig_dti` when it is not missing and otherwise it is imputed with the median. This way, we can use both `dti_missing` and `orig_dti_imp` as variables in our model.

Note: all loans in the data happen to be *fixed* rate mortgages (FRMs) and *not* be prepayment penalty mortgages (PPMs). For this reason, we cannot include the indicators for either of these factors in our model, and it also means that our model is implicitly conditioning on the loan being an FRM and a non-PPM.

Load and Clean Macroeconomic Data

Macroeconomic variables are included in the model as exogenous time-varying covariates, which allows the model to condition on observed (lagged) macroeconomic variables and hence help the model learn the overall economic environment that lenders face at a given point in time.

```
months <- full_df %>%
  select(date) %>%
  unique() %>%
  arrange(date)
months <- months$date

# View first month of time window
months[1]
```

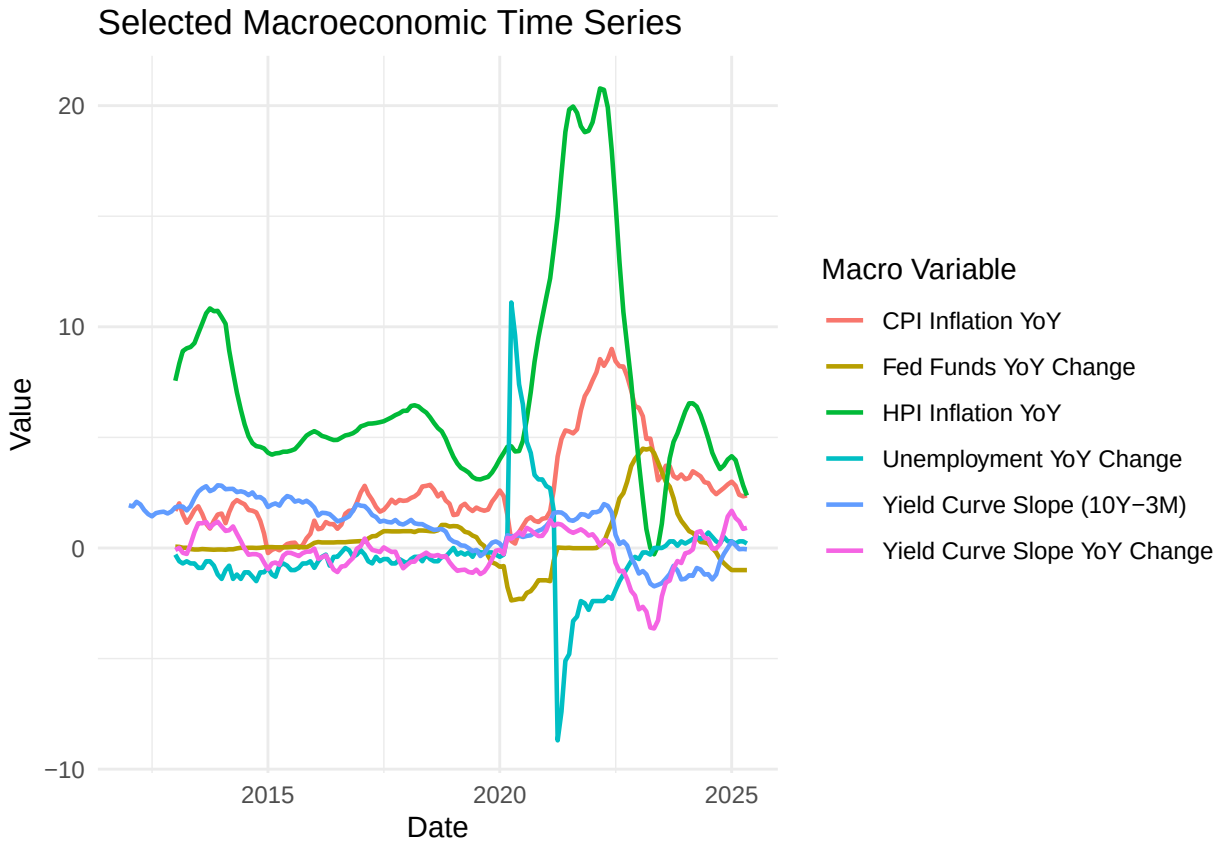
```
## [1] "2013-02-01"
```

```
# Set start and end date for macro variables
start_date <- as.Date("2012-01-01")
end_date <- as.Date(months[length(months)])

pull_fred <- function(series_id, monthly_avg=FALSE) {
  df <- fredr(
      series_id=series_id,
      observation_start=start_date,
      observation_end=end_date
    ) %>%
    select(date, value)
  if (monthly_avg) {
    df %>% mutate(
      ym = floor_date(date, "month") # month bucket
      ) %>%
      group_by(ym) %>%
      summarise(
        value = mean(value, na.rm=TRUE), .groups='drop'
        ) %>%
      rename(date = ym, !!tolower(series_id) := value) # rename value column to the series_id
```

```
  } else {
    df %>%
      rename(!!tolower(series_id) := value) # rename value column to the series_id
  }
}


# Get time series from FRED
unrate <- pull_fred("UNRATE")
cpi <- pull_fred("CPIAUCSL") %>% rename(cpi = cpiaucsl)
fedfunds <- pull_fred("FEDFUNDS")
t10rate <- pull_fred("GS10") %>% rename(t10rate = gs10)
hpi <- pull_fred("CSUSHPINSA") %>% rename(hpi = csushpinsa)
slope <- pull_fred("T10Y3M", monthly_avg=TRUE) %>% rename(slope = t10y3m)
vix <- pull_fred("VIXCLS", monthly_avg=TRUE) %>% rename(vix = vixcls)

# Merge into a single dataframe
macro <- unrate %>%
  left_join(cpi, by='date') %>%
  left_join(fedfunds, by='date') %>%
  left_join(hpi, by='date') %>%
  left_join(slope, by='date') %>%
  left_join(vix, by='date') %>%
  left_join(t10rate, by='date') %>%
  arrange(date) %>%
  mutate(
    # growth in CPI (inflation) YoY
    infl_yoy = 100*(cpi / lag(cpi, 12) - 1),
    # home inflation YoY
    hpi_yoy = 100*(hpi / lag(hpi, 12) - 1),
    # change in UNRATE YoY
    unrate_chg_yoy = unrate - lag(unrate, 12),
    # change in FEDFUNDS YoY
    fedfunds_chg_yoy = fedfunds - lag(fedfunds, 12),
    # change in yield curve slope YoY
    slope_chg_yoy = slope - lag(slope, 12)
  )

# Prep data for plotting
macro_long <- macro %>%
  select(date, infl_yoy, hpi_yoy, unrate_chg_yoy,
         fedfunds_chg_yoy, slope, slope_chg_yoy) %>%
  pivot_longer(cols = -date, names_to = "series", values_to = "value")
macro_long$series <- recode(
  macro_long$series,
  infl_yoy = "CPI Inflation YoY",
  hpi_yoy = "HPI Inflation YoY",
```

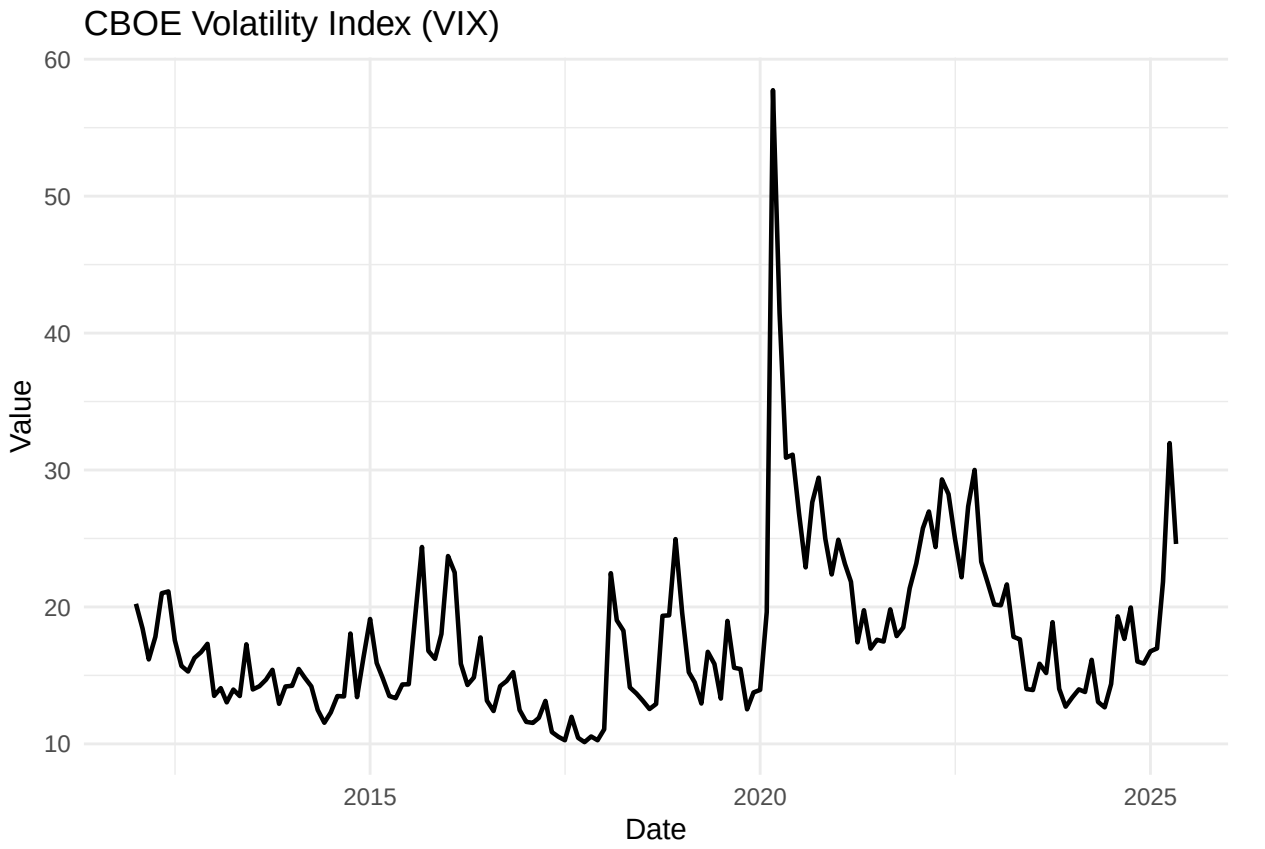
```
  unrate_chg_yoy = "Unemployment YoY Change",
  fedfunds_chg_yoy = "Fed Funds YoY Change",
  slope = "Yield Curve Slope (10Y-3M)",
  slope_chg_yoy = "Yield Curve Slope YoY Change"
)
# Plot all the macro time series
ggplot(macro_long, aes(x = date, y = value, color = series)) +
  geom_line(linewidth = 0.8) +
  theme_minimal() +
  labs(
    x = "Date",
    y = "Value",
    color = "Macro Variable",
    title = "Selected Macroeconomic Time Series"
  )
```

```
## Warning: Removed 60 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

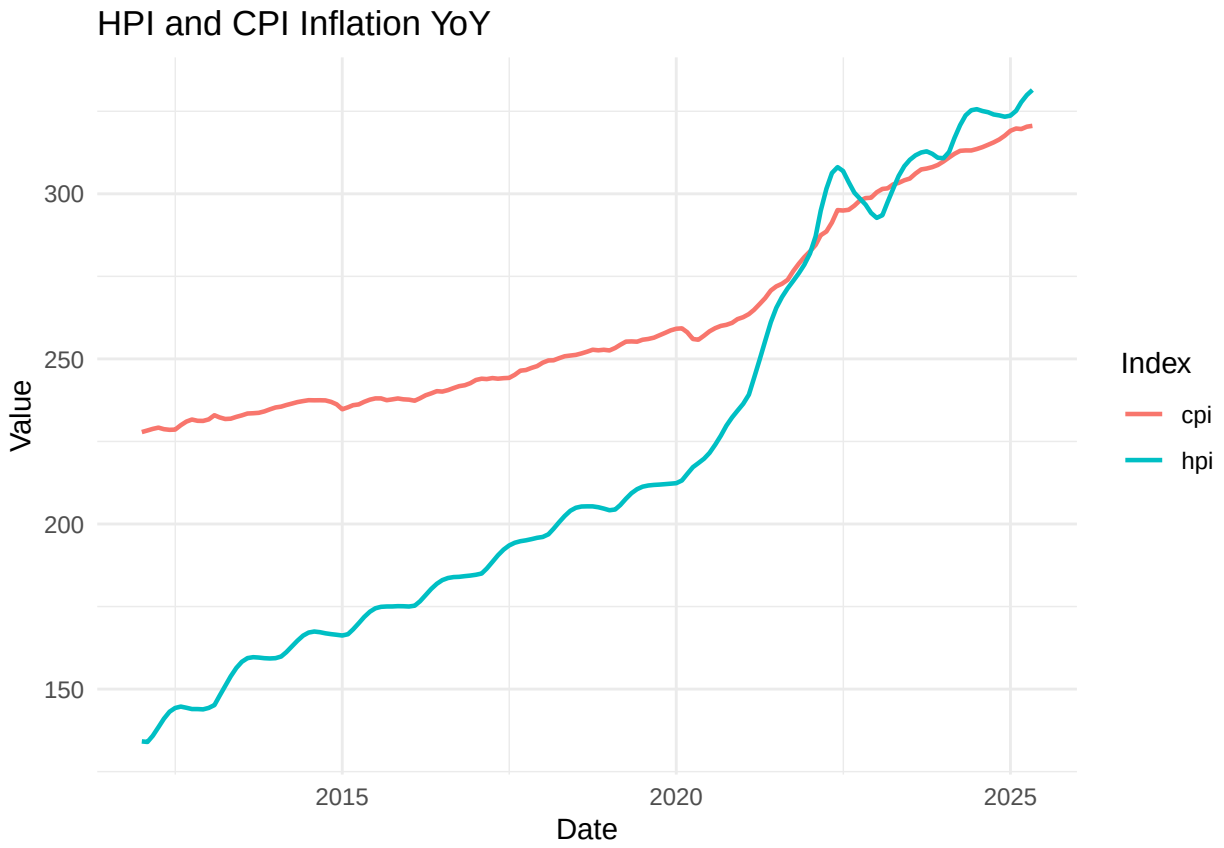



```
rm(macro_long)
ggplot(vix, aes(x = date, y = vix)) +
  geom_line(linewidth = 0.8) +
  theme_minimal() +
  labs(
```

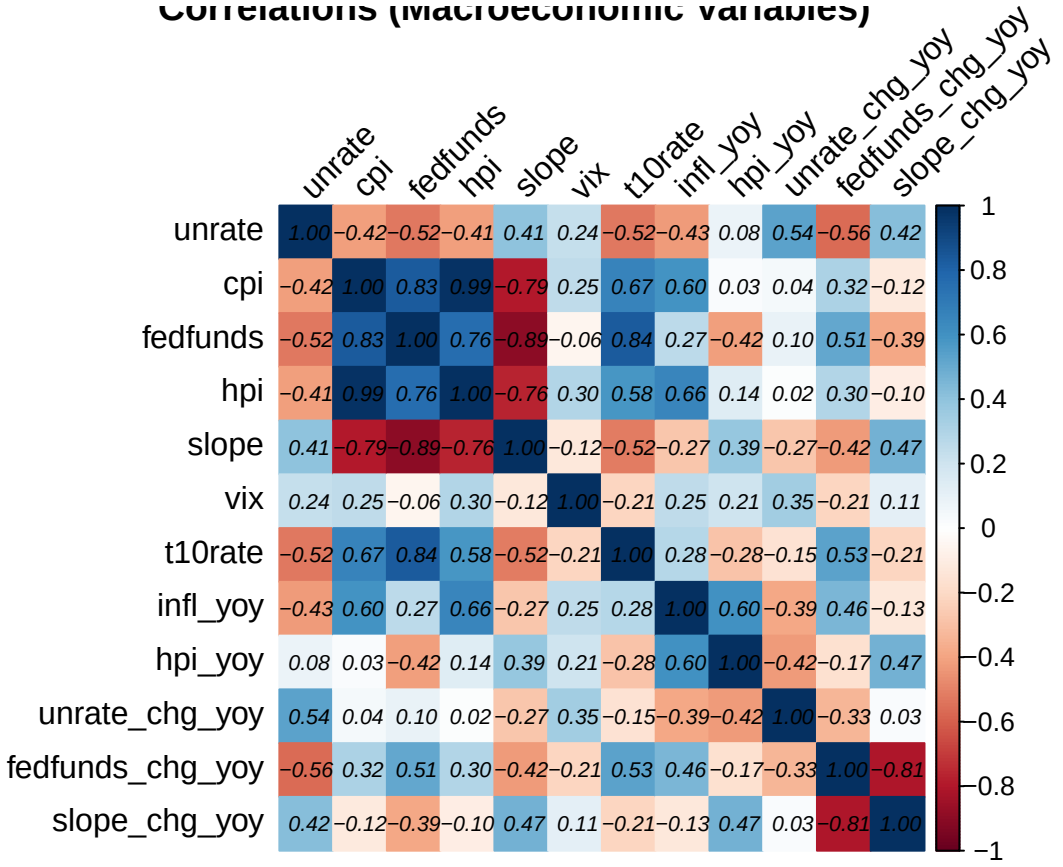
```
    x = "Date",
    y = "Value",
    title = "CBOE Volatility Index (VIX)"
  )
```




```
index_levels <- data.frame(
  date = rep(cpi$date, 2),
  value = c(hpi$hpi, cpi$cpi),
  series = c(rep("hpi",length(cpi$date)), rep("cpi",length(cpi$date)))
)
ggplot(index_levels, aes(x = date, y = value, color = series)) +
  geom_line(linewidth = 0.8) +
  theme_minimal() +
  labs(
    x = "Date",
    y = "Value",
    color = "Index",
    title = "HPI and CPI Inflation YoY"
  )
```




```
macro <- macro %>% filter(!is.na(infl_yoy))
# Correlation matrix heat map for all macroeconomic variables
cor_macro <- macro %>% select(-date) %>% cor()
corrplot(cor_macro, method="color", tl.col="black", addCoef.col="black",
         tl.srt=45, number.font=3, number.cex=0.65,
         title="Correlations (Macroeconomic Variables)")
```




```r
# Lag macro variables 1mo to prevent leakage
macro <- macro %>% mutate(
  last_t10rate = lag(t10rate, 1),
  last_cpi = lag(cpi, 1),
  last_infl_yoy = lag(infl_yoy, 1),
  last_hpi = lag(hpi, 1),
  last_hpi_yoy = lag(hpi_yoy, 1),
  last_unrate_chg_yoy = lag(unrate_chg_yoy, 1),
  last_fedfunds = lag(fedfunds, 1),
  last_fedfunds_chg_yoy = lag(fedfunds_chg_yoy, 1),
  # slope and vix are still lagged because we use monthly averages
  last_slope = lag(slope, 1),
  last_slope_chg_yoy = lag(slope_chg_yoy, 1),
  last_vix = lag(vix, 1)
) %>%
  select(
   date, last_t10rate, last_cpi, last_infl_yoy, last_hpi_yoy, last_hpi,
   last_vix, last_unrate_chg_yoy, last_fedfunds, last_fedfunds_chg_yoy,
   last_slope, last_slope_chg_yoy
  )

# Merge macro data to loan-month data
full_df <- full_df %>%
```

```r
  mutate(date = as.Date(date))
full_df <- full_df %>%
  left_join(macro, by="date")

# Derive date of origination
loan_vintages <- full_df %>%
  group_by(loan_sequence_number) %>%
  summarise(
    entry_year = year(min(date)),
    .groups = "drop"
  ) %>%
  mutate(
    vintage_year = case_when(
      entry_year %in% 2013:2015 ~ 2013,
      entry_year %in% 2016:2019 ~ 2016,
      entry_year %in% 2020:2025 ~ 2020,
      TRUE ~ entry_year
    )
  )
orig_df$vintage_year <- NULL
orig_df <- orig_df %>%
  left_join(loan_vintages %>% select(loan_sequence_number, vintage_year),
            by = "loan_sequence_number")
orig_df <- orig_df %>%
  mutate(
    orig_quarter = case_when(
      quarter == "Q1" ~ 1,
      quarter == "Q2" ~ 2,
      quarter == "Q3" ~ 3,
      quarter == "Q4" ~ 4
    ),
    orig_month = case_when(
      orig_quarter == 1 ~ 1, # Jan
      orig_quarter == 2 ~ 4, # Apr
      orig_quarter == 3 ~ 7, # Jul
      orig_quarter == 4 ~ 10 # Oct
    ),
    orig_date = as.Date(sprintf("%d-%02d-01", vintage_year, orig_month))
  )
full_df <- full_df %>%
  left_join(orig_df %>% select(loan_sequence_number, orig_date),
            by = "loan_sequence_number")

# Approximate home price appreciation since origination ("HPAO")
hpi_df <- hpi %>% rename(orig_date = date, hpi_orig = hpi)
full_df$hpi_orig <- NULL
rm(hpi)
```

```r
full_df <- full_df %>%
  left_join( # join HPI at origination
    hpi_df, by = "orig_date") %>%
  mutate(
    last_hpao = 100 * (last_hpi / hpi_orig - 1)
  )
rm(loan_vintages)



## Try hpi_yoy, hpao, orig_ltv, and orig_ltv * I(1/hpao) in the model.
```

It is necessary to lag the macroeconomic features by (at least) one month because the data for month $t-1$ is typically only released during month t.

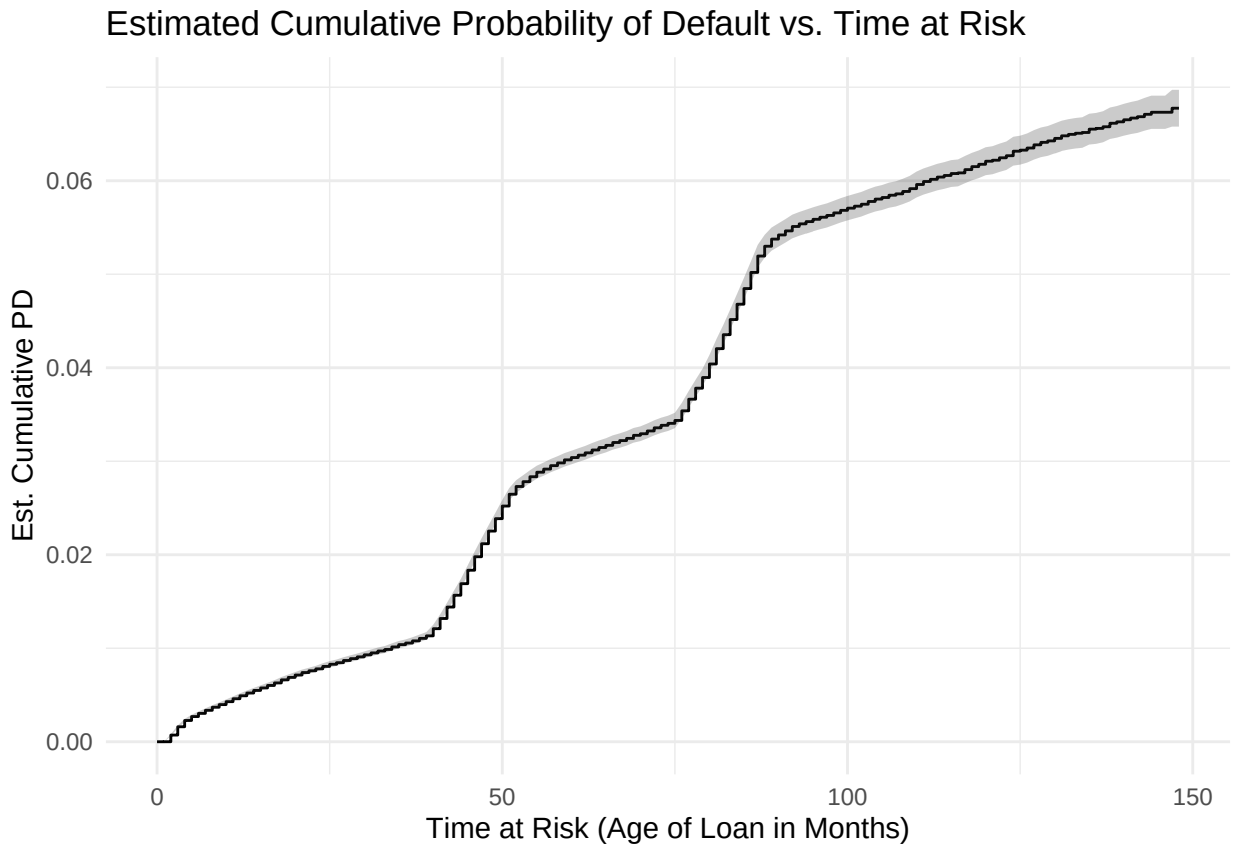
We could also try some different seasonal growth rates or differences for the macroeconomic covariates, such as a more short-term feature for changes in the Federal Funds rate (e.g. quarterly difference), although for the sake of this project we stick to the features derived above.

EDA, Variable Selection and Feature Engineering

Univariate Survival Analysis

```r
library(survival) # for KM estimator

# Aggregate (loan, month)-indexed binary data into loan-indexed count data
surv_df <- full_df %>%
  group_by(loan_sequence_number) %>%
  arrange(date, .by_group = TRUE) %>%
  mutate(t = loan_age) %>%
  summarise(
    time = ifelse(any(y==1), yes=min(t[y==1]), no=max(t)), # event or censor time
    status = as.integer(any(y == 1)), # 1 = observed event ; 0 = censored
    .groups = "drop"
  )

# Fit KM curve (estimate survival function)
fit <- survfit(Surv(time, status) ~ 1, data = surv_df)

rm(surv_df)

# Estimate CDF
cdf_fit <- data.frame(time = fit$time, surv = fit$surv,
                      lower = 1 - fit$lower, upper = 1 - fit$upper) %>%
  mutate(cdf = 1 - surv) %>%
  select(-surv)
```

```r
rm(fit)

# Plot CDF estimate with confidence limits
ggplot(cdf_fit, aes(x=time, y=cdf)) +
  geom_step() +
  geom_ribbon(aes(ymin=lower, ymax=upper), alpha=0.25) +
  theme_minimal() +
  labs(title="Estimated Cumulative Probability of Default vs. Time at Risk",
       x="Time at Risk (Age of Loan in Months)",
       y="Est. Cumulative PD")
```

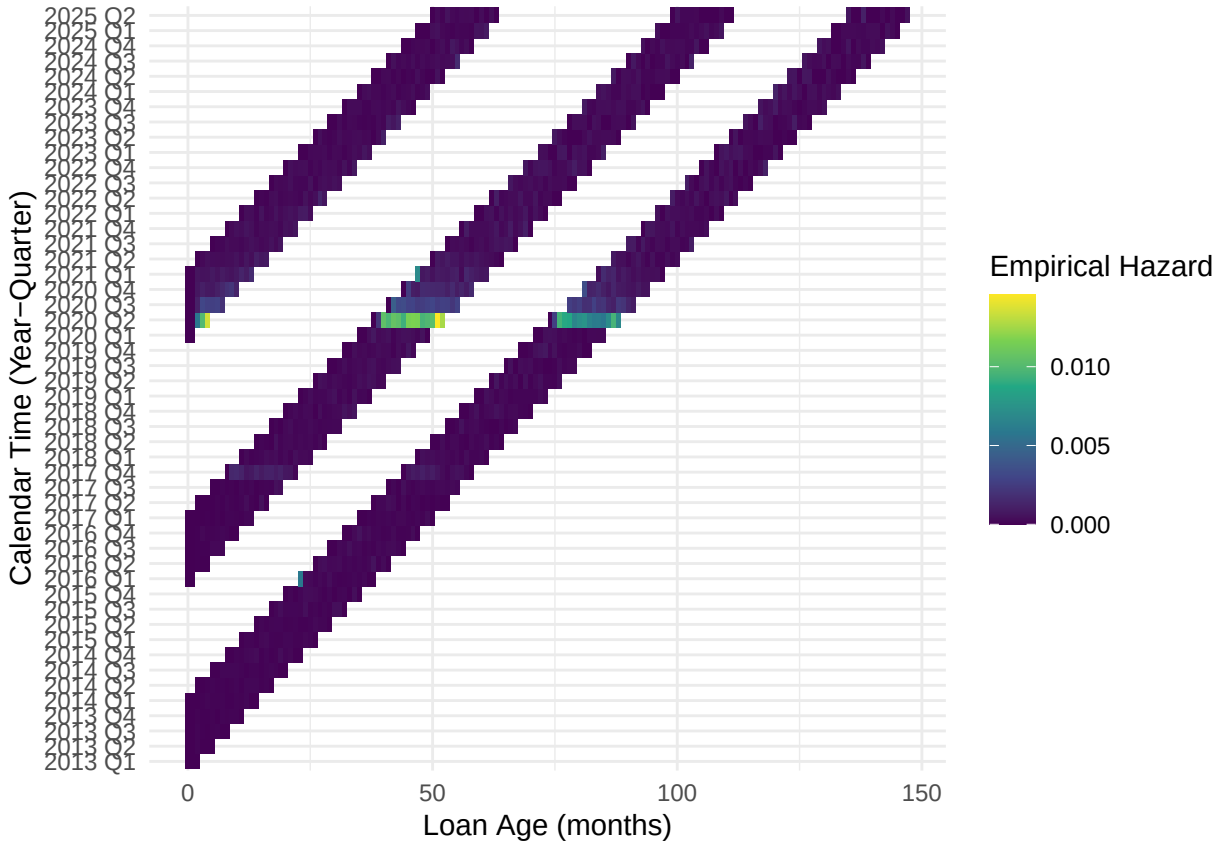


The estimated cumulative PD (eCPD) curve shows spikes at around 40-50 months and 75-90 months, which is suspected to be the effects of macroeconomic shocks from those periods of time (i.e. COVID). For the 2016 origintion sample, a loan age of 40-50 months roughly corresponds to the time period of 2020 Q2 on average, and similarly for the 2013 origination sample, a loan age of 75-90 months roughly corresponds to the time period of 2020 Q2 on average. If we look more closely at the very start of the eCPD curve for small loan ages, we see that it is very subtly increasing at a faster pace than the prolonged steady rates of growth we see for later months, which aligns with the fact that the 2020 origination sample was in the calendar time period of 2020 Q1-Q2 for these loan ages.

The goal is to use macroeconomic covariates such as the unemployment rate and the house price index in our PD model so that we can try to capture the effects of macro shocks on PD and so that we can identify the possible effect of other variables like loan age without the issue of confounding.

Identifying Possible Age of Loan Effects

```
full_df <- full_df %>%
  group_by(loan_sequence_number) %>%
  arrange(date, .by_group = TRUE) %>%
  ungroup()

full_df <- full_df %>% mutate(
  yq = paste(format(date, "%Y"), quarters(date))
)

haz_age_date <- full_df %>%
  group_by(loan_age, yq) %>%
  summarise(
    haz = mean(y),
    n = n(), .groups = "drop")

# Plot calendar time and age of loan simultaneously
ggplot(haz_age_date %>% filter(n > 100),
       aes(x = loan_age, y = yq, fill = haz)) +
  geom_tile() +
  scale_fill_viridis_c() +
  labs(
    x = "Loan Age (months)",
    y = "Calendar Time (Year-Quarter)",
    fill = "Empirical Hazard"
  ) +
  theme_minimal()
```




```
rm(haz_age_date)
```

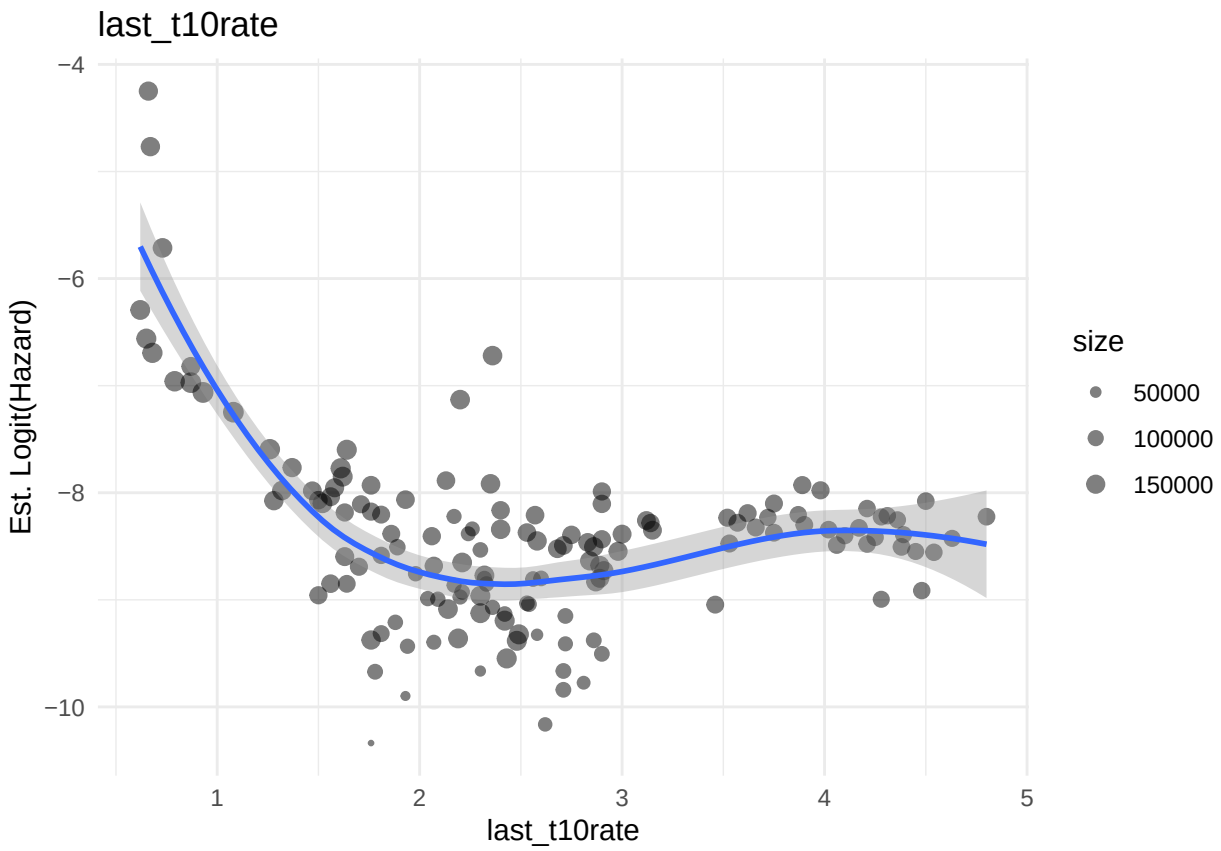
From the heat map above there does not seem to be any association between hazard and loan age. However, we do see clear spikes in hazard across loans of *all ages* during the time period of 2020 Q2. Thus, it seems that macroeconomic conditions play an important role in the event of default and age of loan does not, which makes sense economically and aligns with empirical observations in the literature[3].

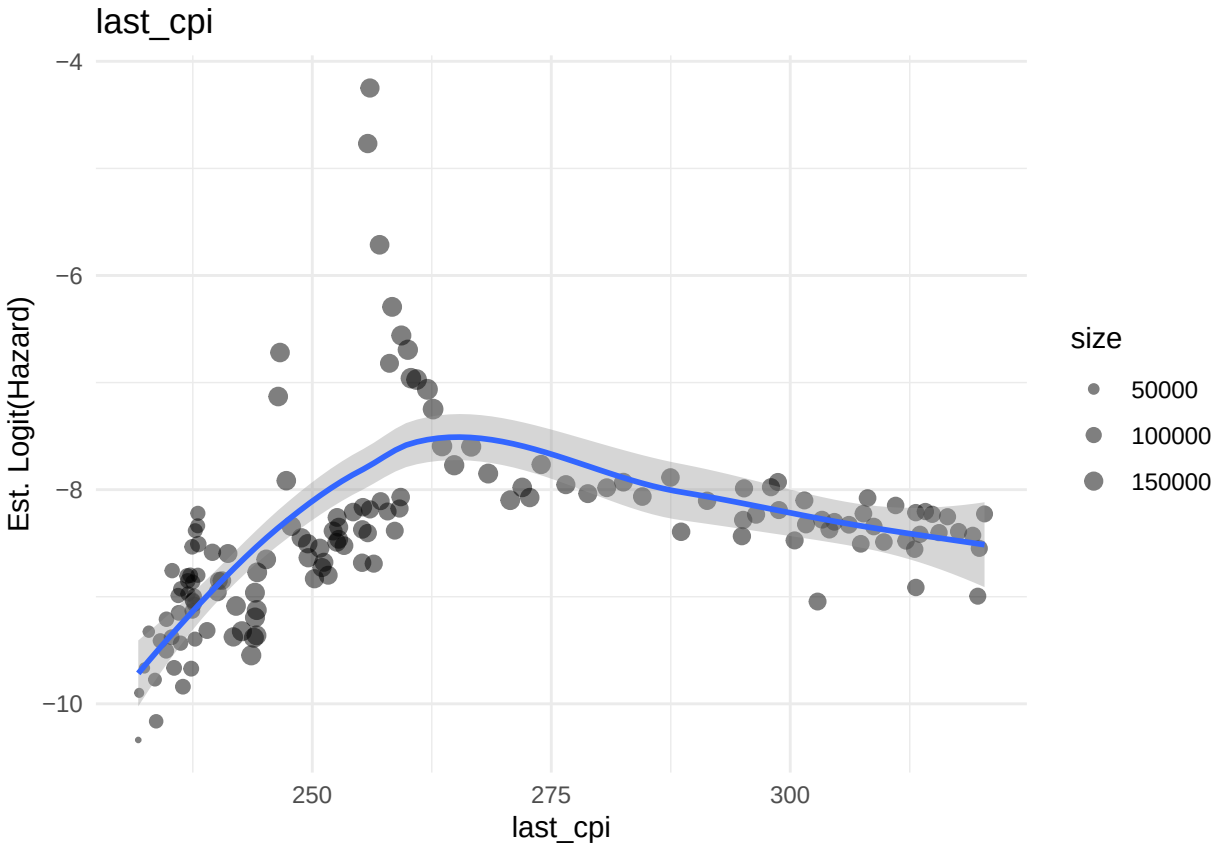
Selecting and Transforming Macroeconmic Features

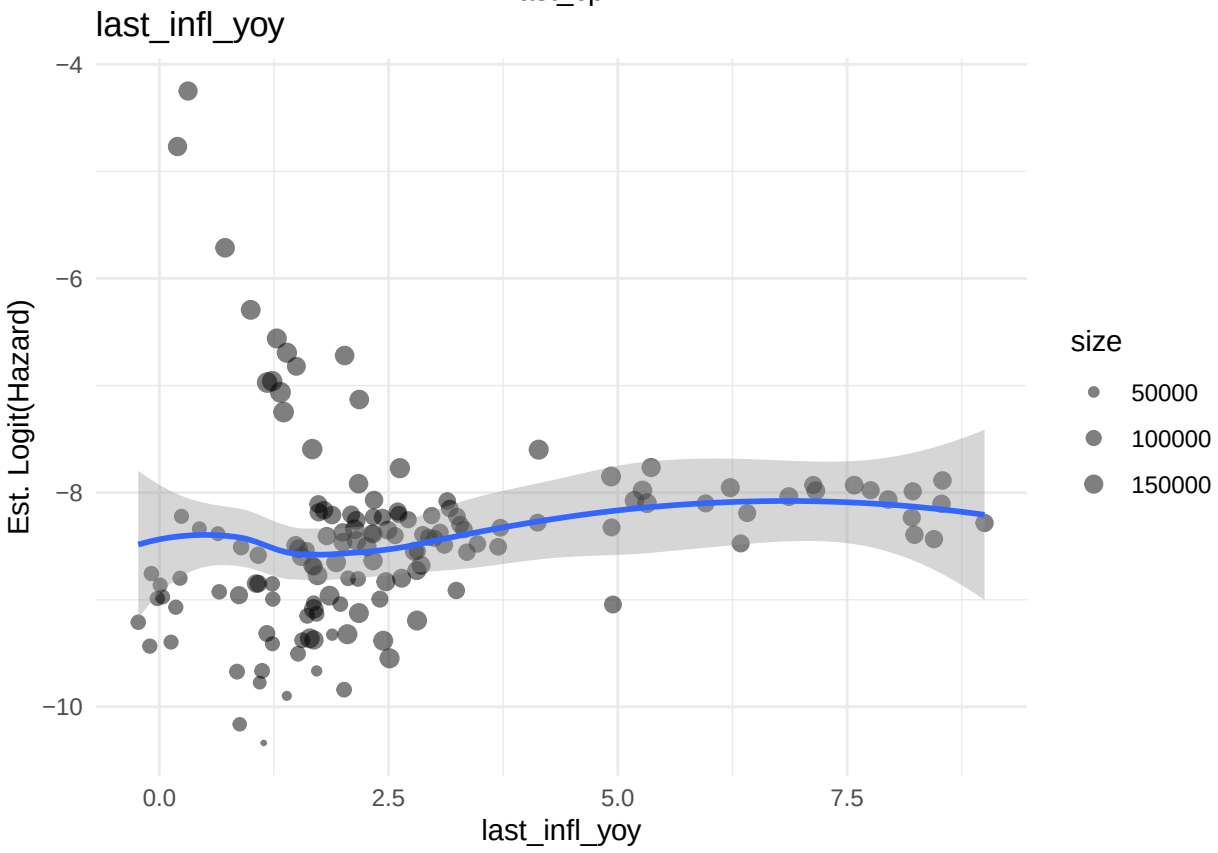
```
haz_date <- full_df %>%
  group_by(date) %>%
  summarise(haz = mean(y), n_obs = n(), .groups = "drop") %>%
  left_join(macro, by="date")

macro_vars <- setdiff(names(macro), "date")
for (var_name in macro_vars) {
  p <- ggplot(haz_date %>% filter(haz > 0 & haz < 1),
              aes(.data[[var_name]], qlogis(haz))) +
        geom_point(aes(size=n_obs), alpha=0.5) +
        geom_smooth(se = TRUE, method = "loess", formula = "y~x") +
        theme_minimal() +
```

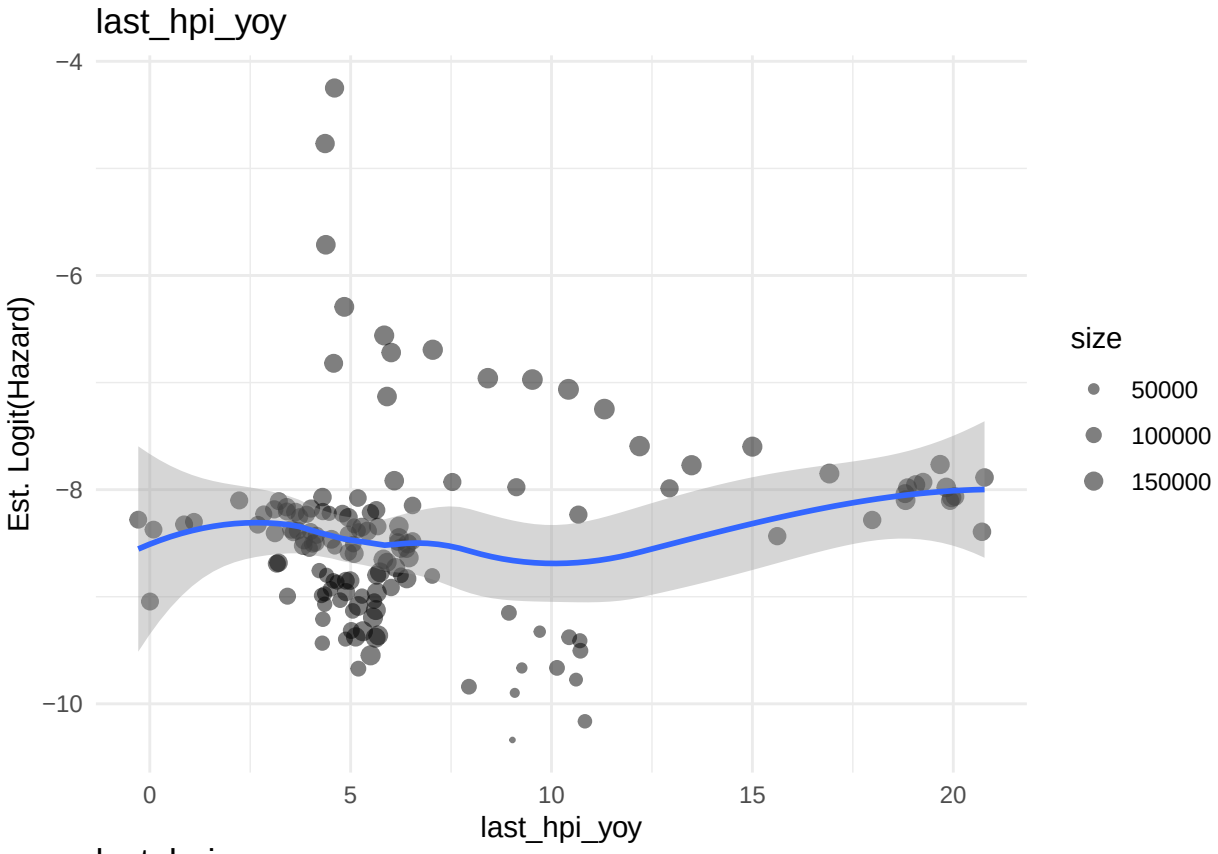
[3]*cite source about time confounding with macro env in PD modeling*

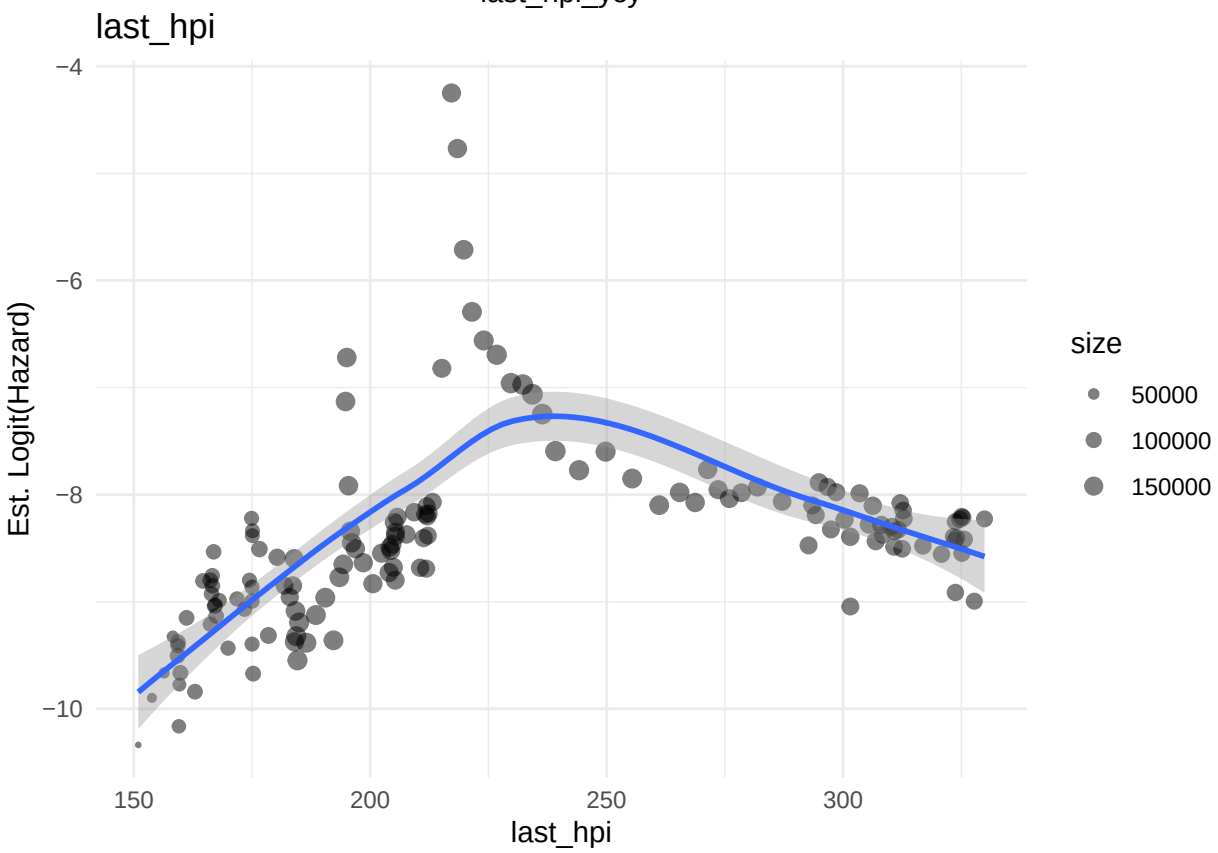
```
        labs(y = "Est. Logit(Hazard)", x = var_name,
             title = var_name) +
    scale_size_continuous(name ="size", range=c(0.5, 3))
  print(p)
}
```

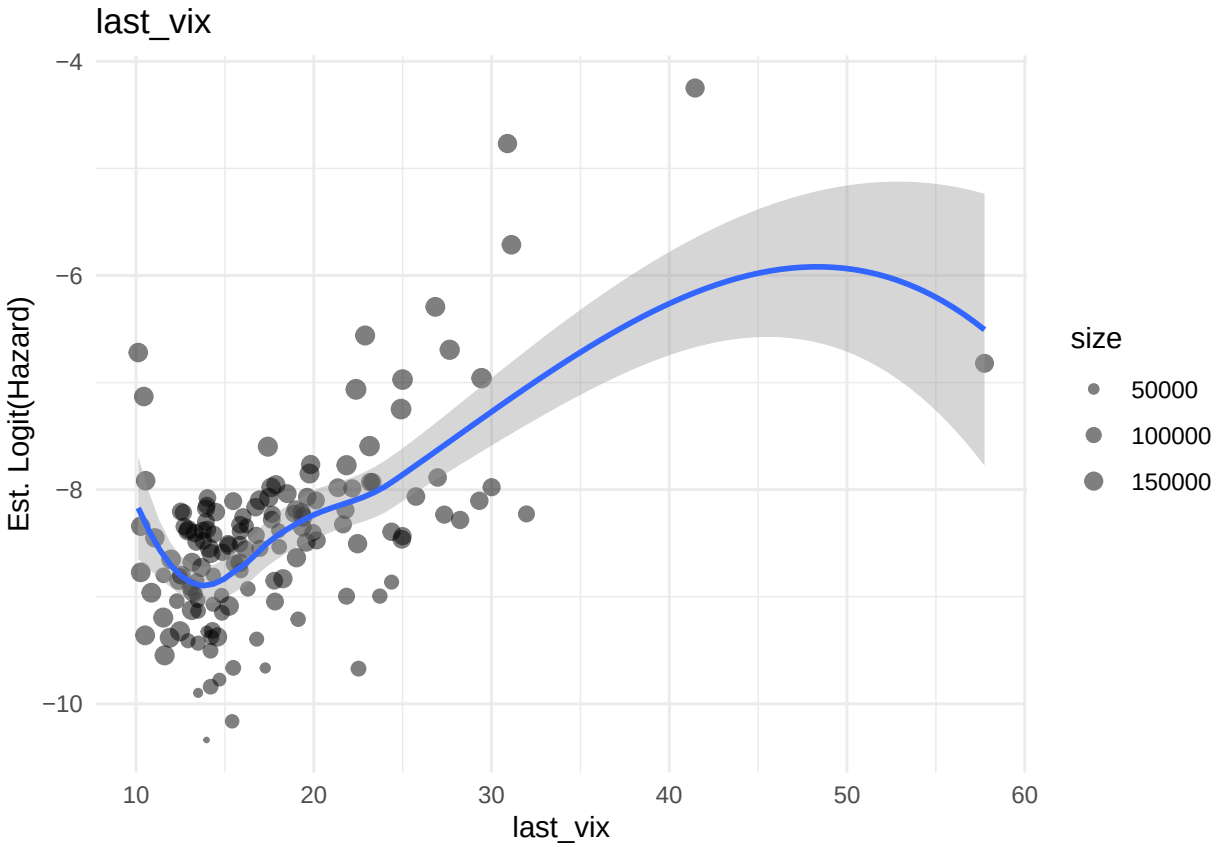


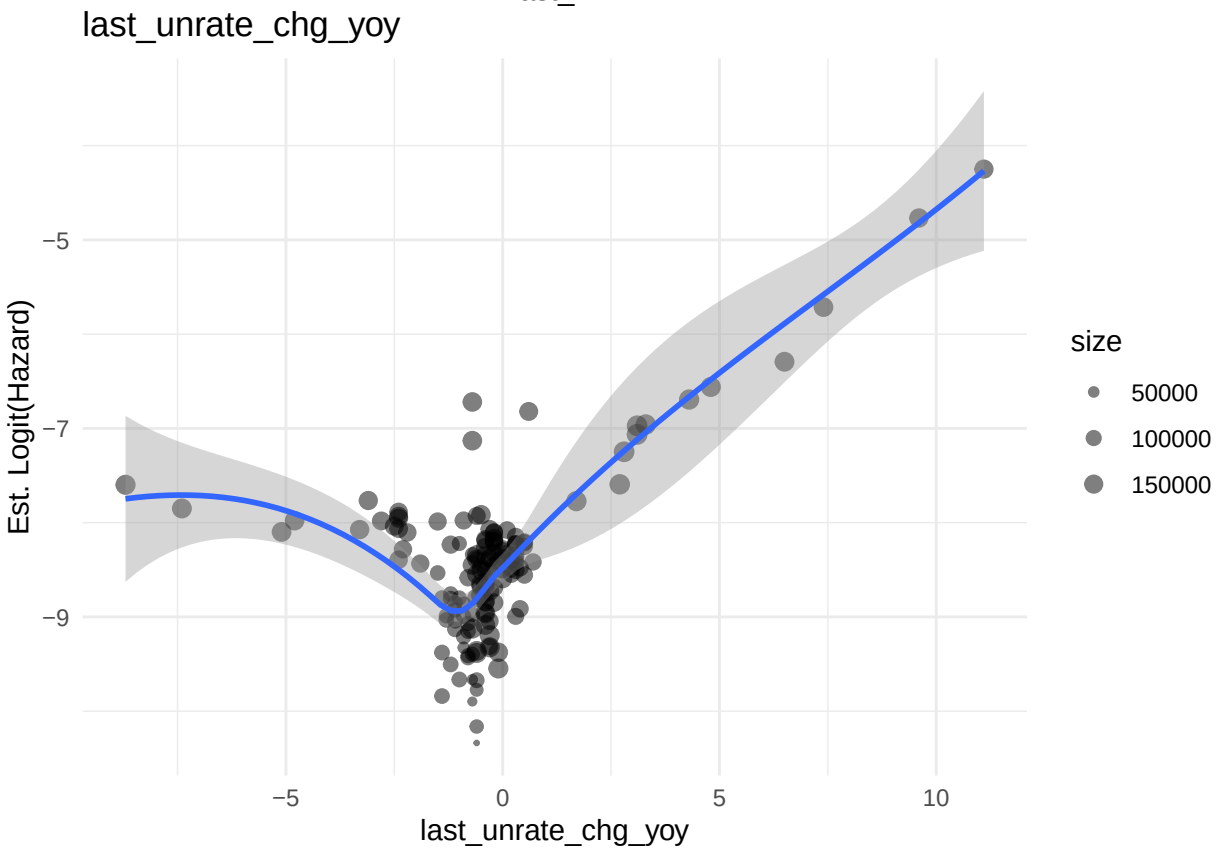

last_cpi
Est. Logit(Hazard)
−4
−6
−8
−10
250
275
300
last_cpi
size
50000
100000
150000

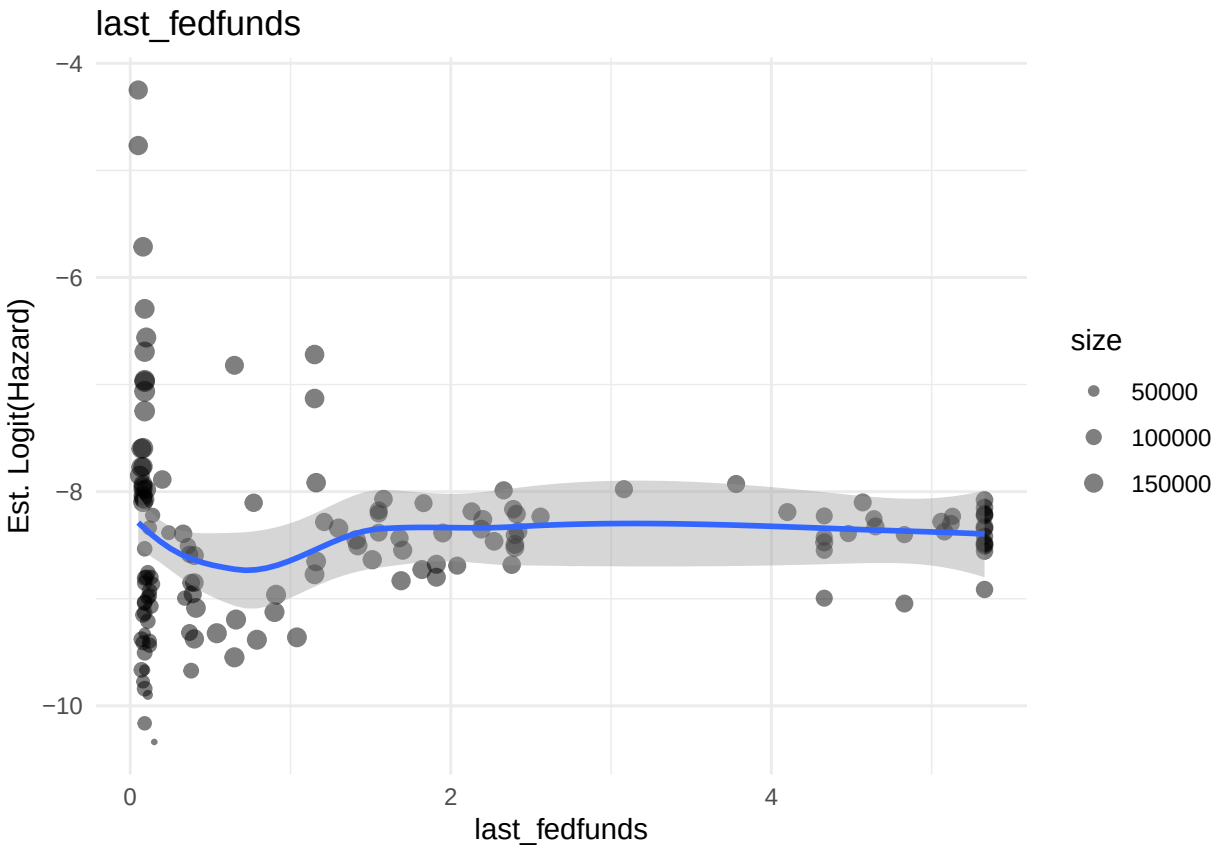

last_infl_yoy
Est. Logit(Hazard)
−4
−6
−8
−10
0.0
2.5
5.0
7.5
last_infl_yoy
size
50000
100000
150000

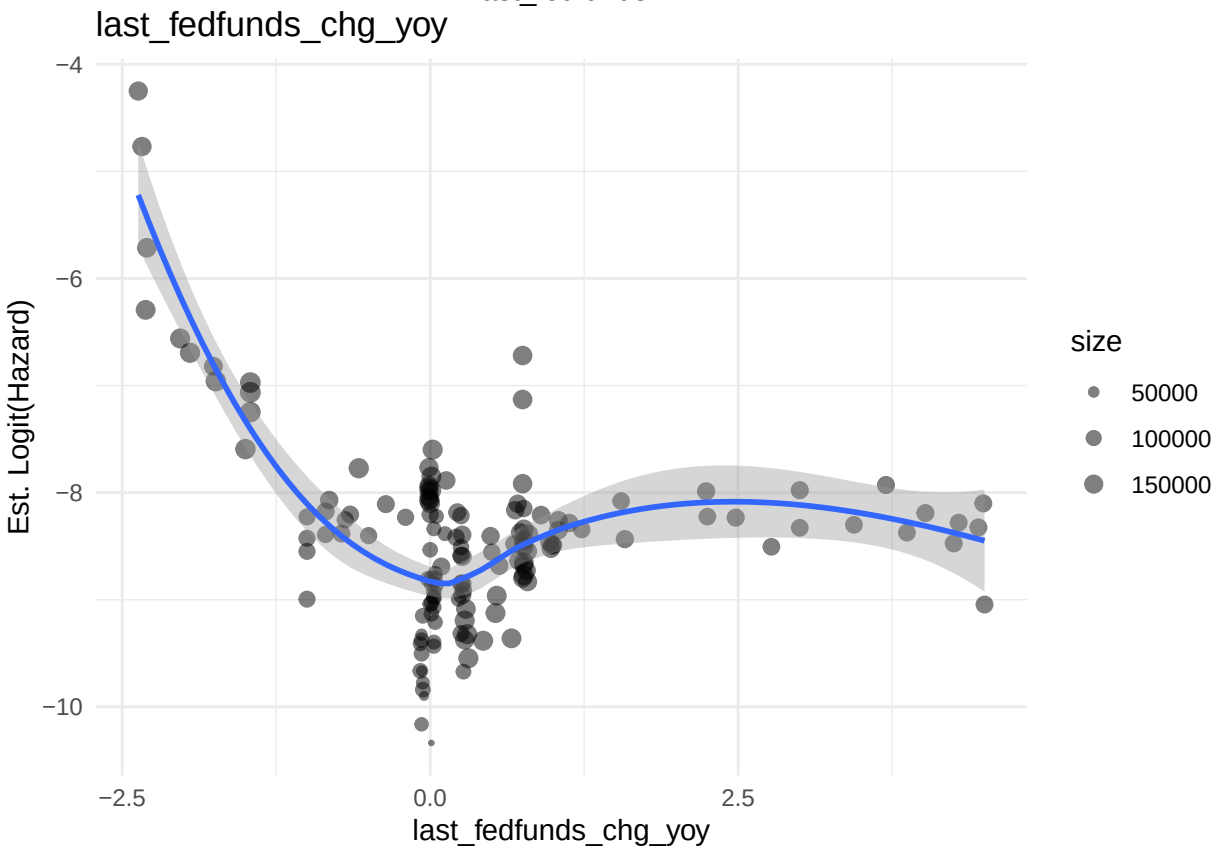

last_hpi_yoy
Est. Logit(Hazard)
−4
−6
−8
−10
0
5
10
15
20
last_hpi_yoy
size
50000
100000
150000

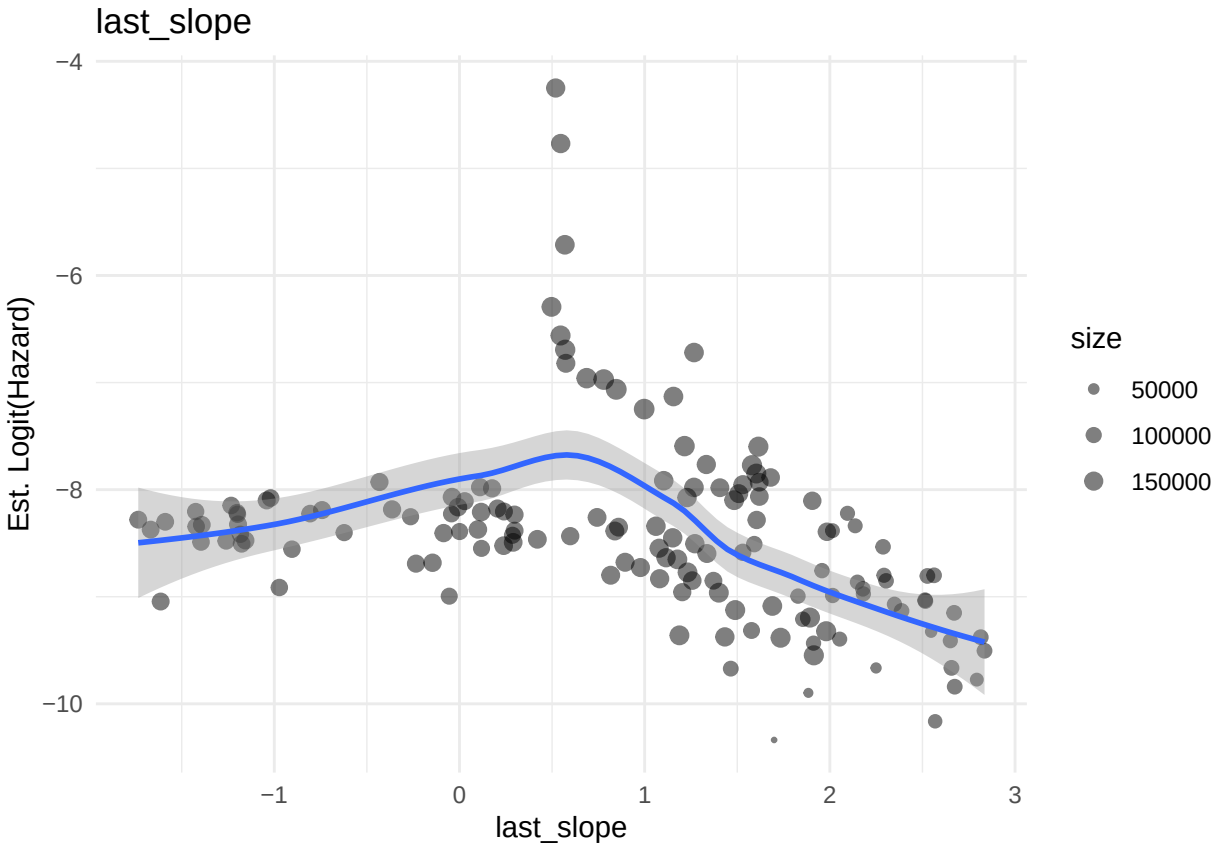

last_hpi
Est. Logit(Hazard)
−4
−6
−8
−10
150
200
250
300
last_hpi
size
50000
100000
150000

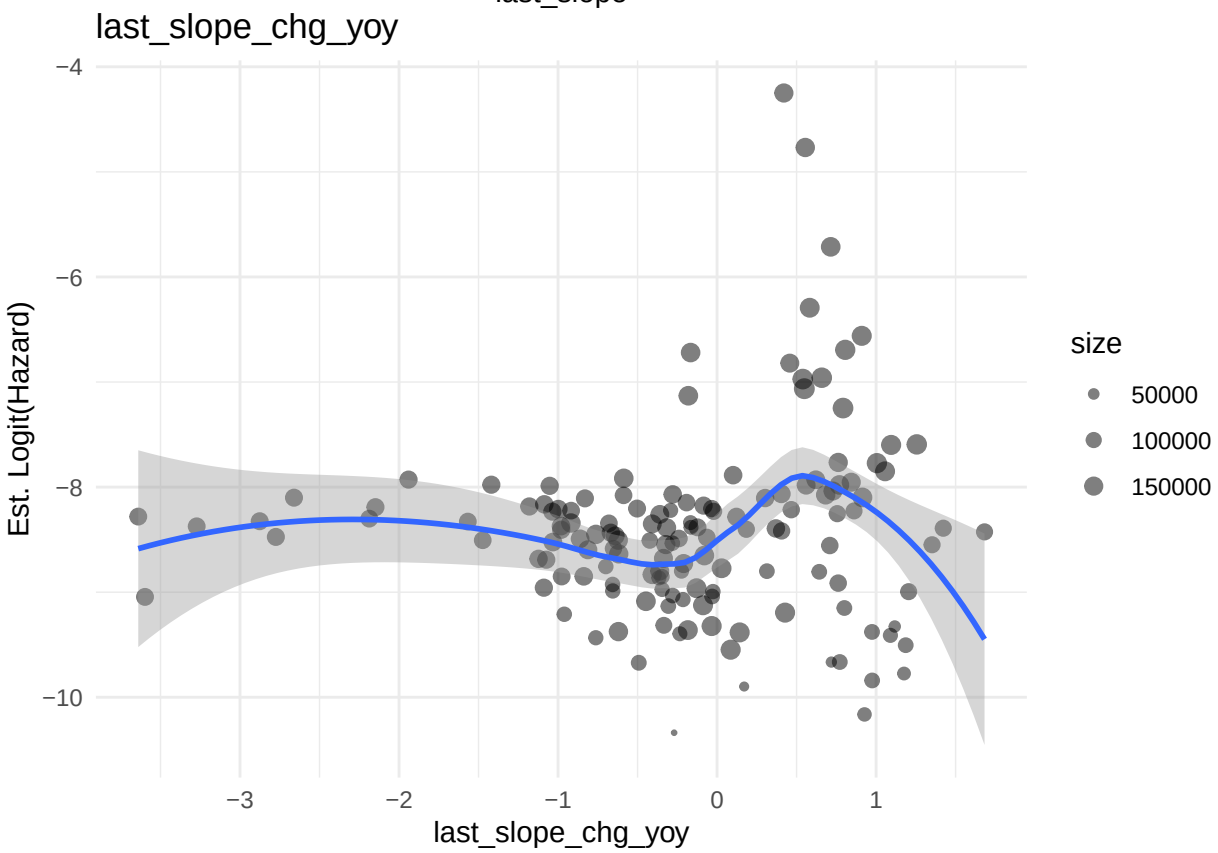

last_vix
Est. Logit(Hazard)
−4
−6
−8
−10
10
20
30
40
50
60
last_vix
size
50000
100000
150000


last_unrate_chg_yoy
Est. Logit(Hazard)
−5
−7
−9
−5
0
5
10
last_unrate_chg_yoy
size
50000
100000
150000

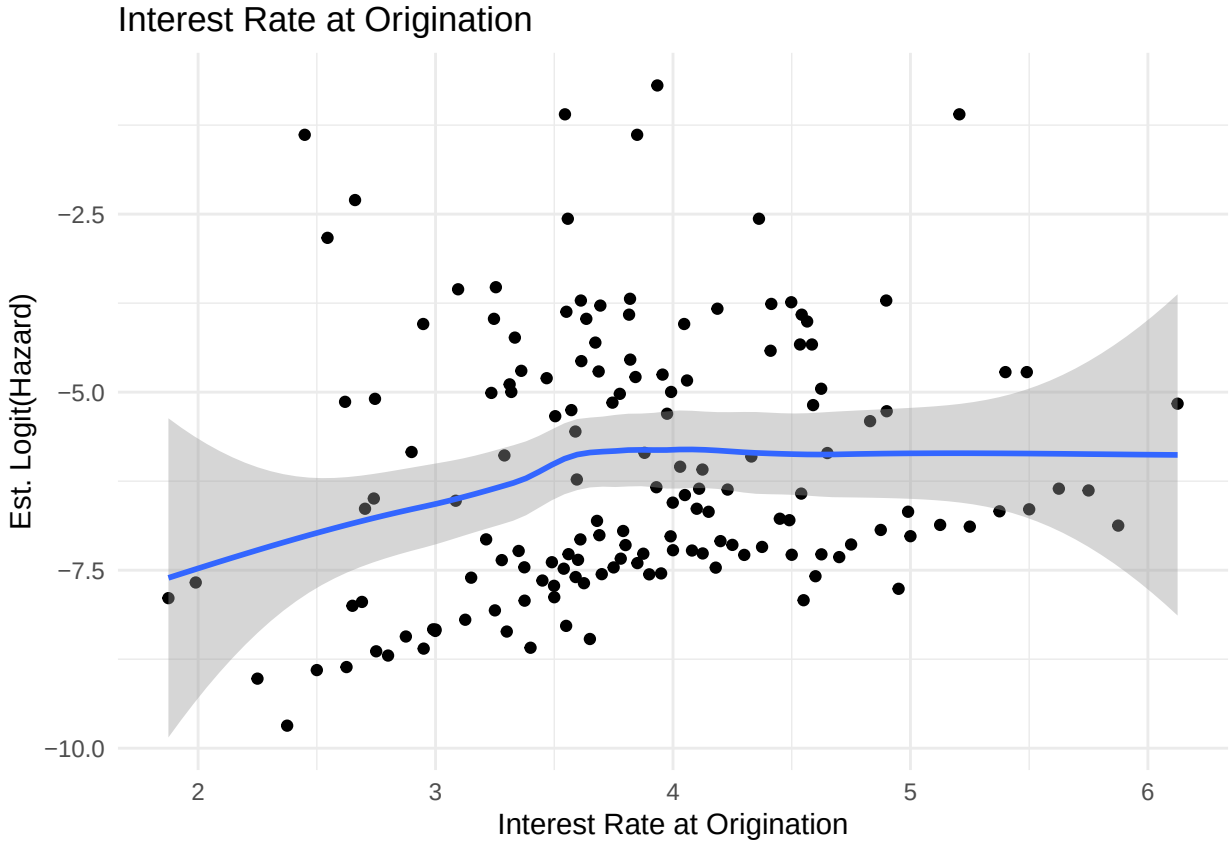

last_fedfunds
Est. Logit(Hazard)
−4
−6
−8
−10
0
2
4
last_fedfunds
size
50000
100000
150000


last_fedfunds_chg_yoy
Est. Logit(Hazard)
−4
−6
−8
−10
−2.5
0.0
2.5
last_fedfunds_chg_yoy
size
50000
100000
150000


last_slope
Est. Logit(Hazard)
−4
−6
−8
−10
−1
0
1
2
3
last_slope
size
50000
100000
150000


last_slope_chg_yoy
Est. Logit(Hazard)
−4
−6
−8
−10
−3
−2
−1
0
1
last_slope_chg_yoy
size
50000
100000
150000

```r
rm(haz_date)

haz_rate <- full_df %>%
  group_by(orig_rate) %>%
  summarise(
    haz = mean(y), .groups = "drop"
  )
p <- ggplot(haz_rate %>% filter(haz > 0 & haz < 1),
            aes(orig_rate, qlogis(haz))) +
        geom_point() +
        geom_smooth(se = TRUE, method = "loess", formula = "y~x") +
        theme_minimal() +
        labs(y = "Est. Logit(Hazard)", x = "Interest Rate at Origination",
             title = "Interest Rate at Origination")
print(p)
```



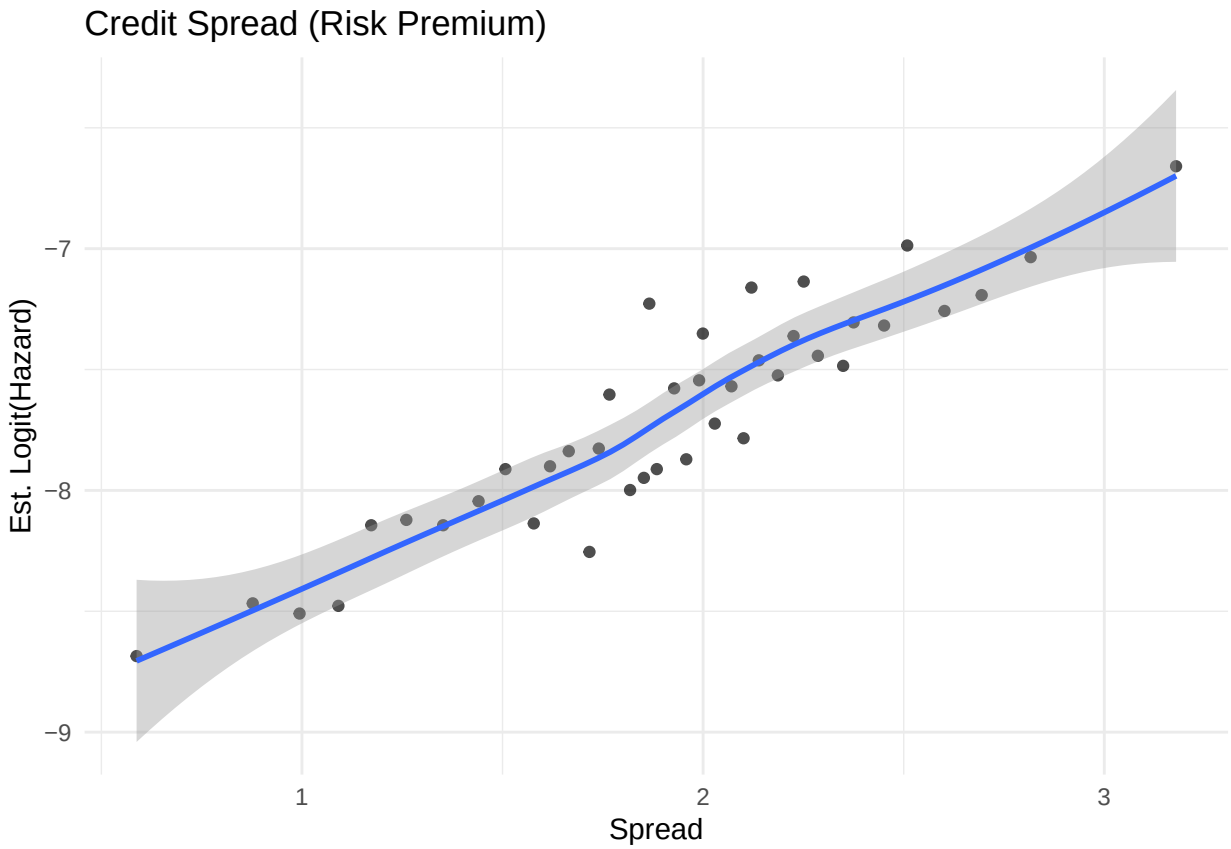

```r
rm(haz_rate)
```

- figuring out how to include raw CPI and HPI levels (or their growth series) is challenging
  - doesnt seem possible to get something out of this while keeping the series properly lagged...
  - these are also strongly colinear with time so might be best to keep them out
- inflation looks like threshold effect: try using min(last_infl_yoy, 2.5)
- change in UR YoY truly looks amazingly linear except with a threshold effect: try using

last_unrate_incr_yoy = last_unrate_chg_yoy * (last_unrate_chg_yoy > 0)

- rate cuts (negative fedfunds change) are really the only significant part so encode that as a feature: last_rate_cut = -last_fedfunds_chg_yoy * (last_fedfunds_chg_yoy < 0) # note the negative sign
- change in yield curve slope: try indicator for it being positive
- for yield curve slope on its own: try last_invert = (last_slope < 0) and if that doesnt work then drop it
- use linear and splines or log or quadratic or linear + excess above 25/30 for VIX then compare via CV/deviance test

```
# Compute credit spread (risk premium) for each loan
orig_df <- orig_df %>%
  left_join(t10rate %>% rename(orig_date = date, t10r = t10rate),
            by="orig_date")
orig_df <- orig_df %>%
  mutate(cr_spread = pmax(orig_interest_rate - t10r, 0))
full_df <- full_df %>%
  left_join(orig_df %>% select(loan_sequence_number, cr_spread),
            by="loan_sequence_number")

# Plot logit hazard vs. credit spread
haz_spread_bin <- full_df %>%
  mutate(cr_bin = ntile(cr_spread, 40)) %>%
  group_by(cr_bin) %>%
  summarise(cr_mu = mean(cr_spread), haz = mean(y), n = n(), .groups = 'drop')
ggplot(haz_spread_bin %>% filter(haz > 0 & haz < 1),
       aes(cr_mu, qlogis(haz))) +
  geom_point(alpha=0.7) +
  geom_smooth(se = TRUE, method = "loess", formula = "y~x") +
  theme_minimal() +
  labs(y = "Est. Logit(Hazard)", x = "Spread",
       title = "Credit Spread (Risk Premium)")
```




```
rm(haz_spread_bin)
```

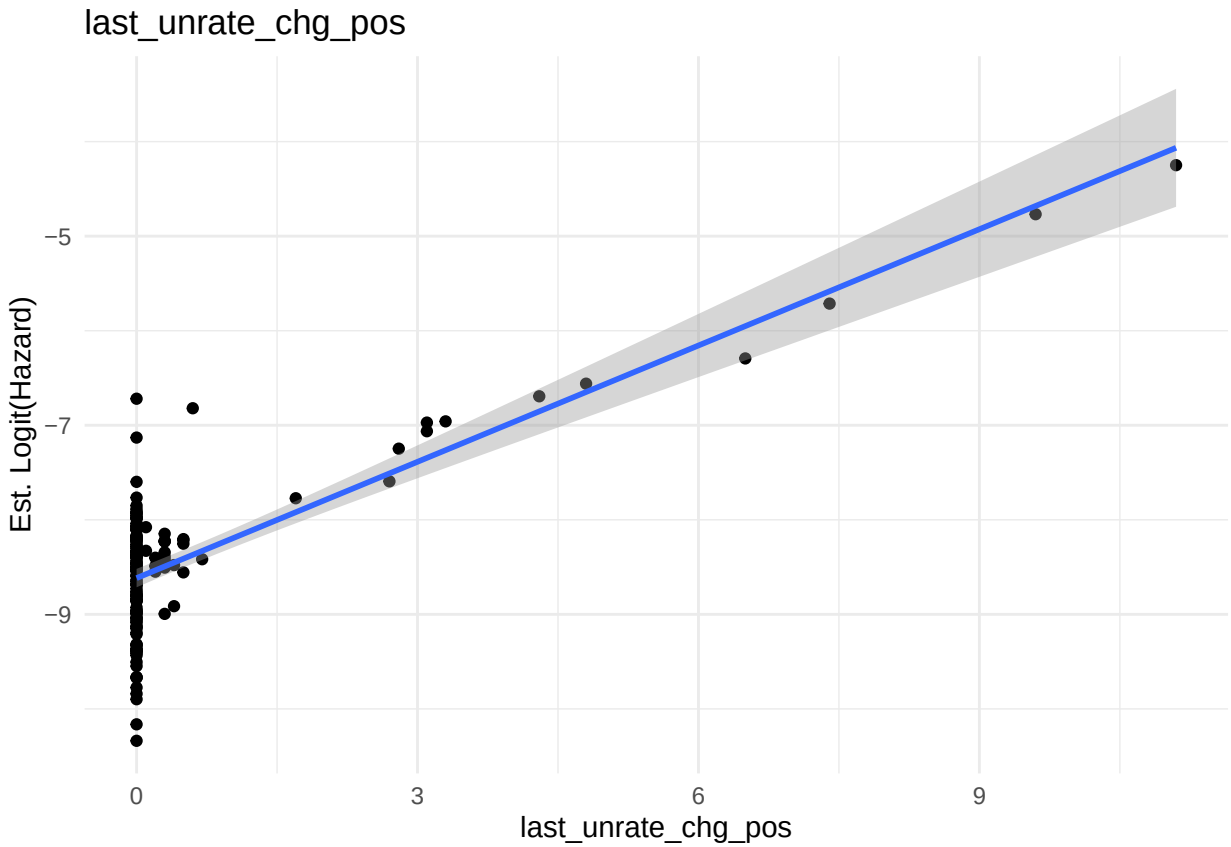
The credit spread shows very clear linear association with the empirical logit hazard.

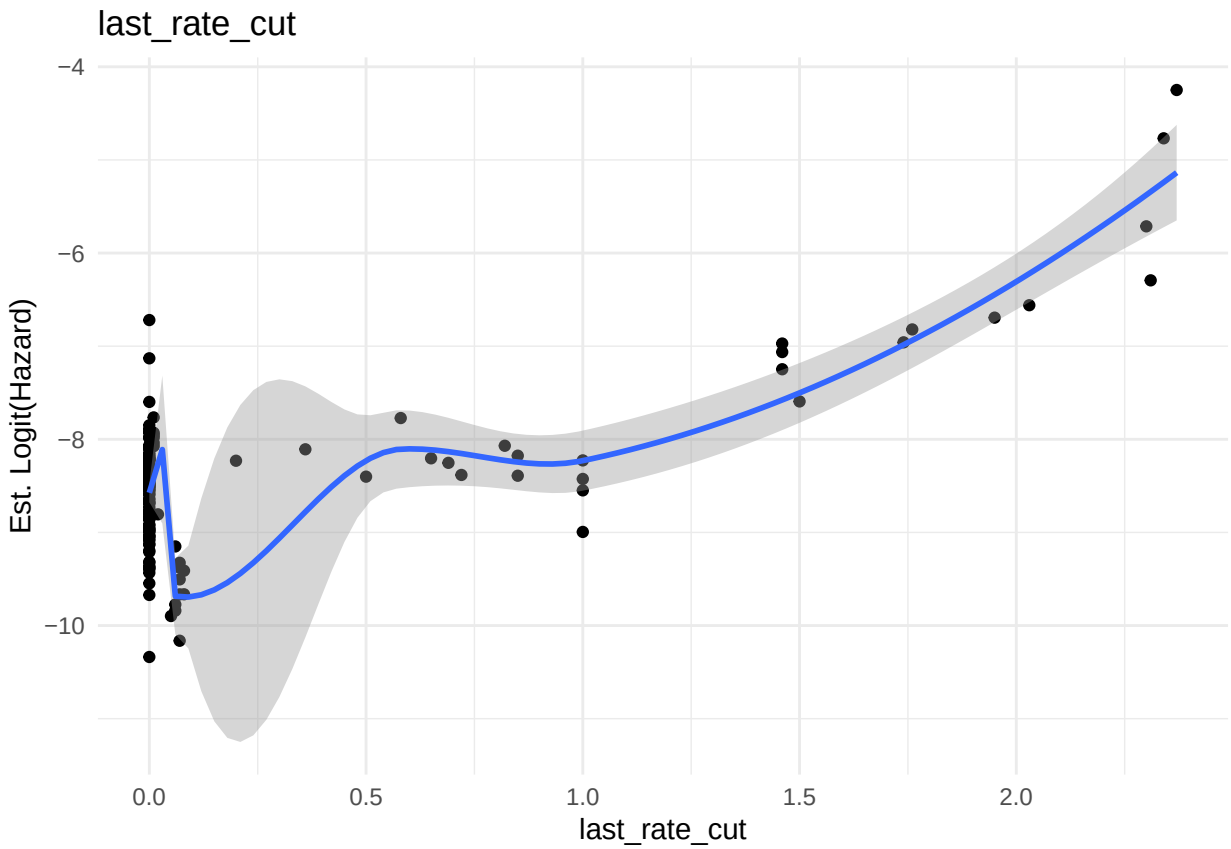
Since the credit spread variable is a linear combination of the interest rate on the loan and the 10-year tresury rate (which is very highly correlated with the federal funds rate), we include the credit spread but not the others.

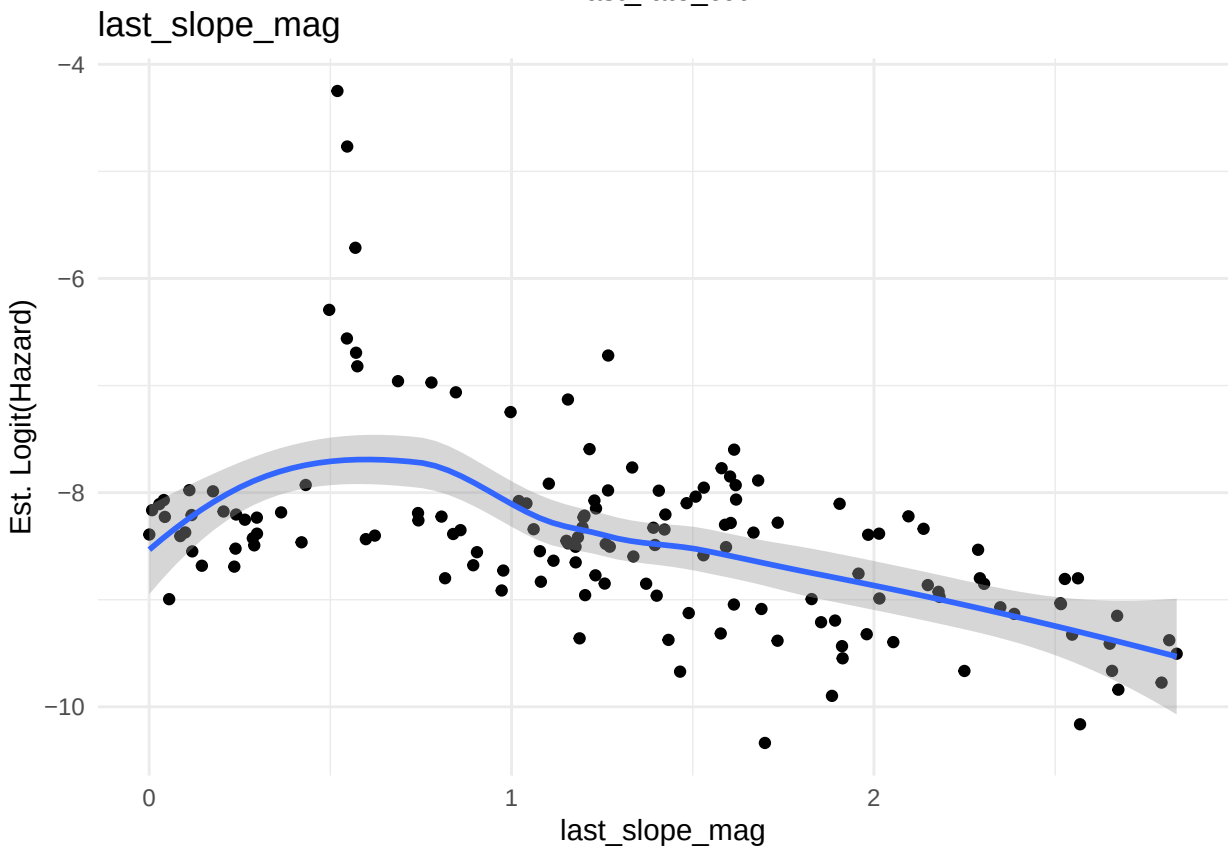
```
# Indicator for low inflation
full_df$last_infl_yoy_low <- factor(as.integer(full_df$last_infl_yoy < 0))
#full_df <- full_df %>% select(-last_infl_yoy)

# Positive part of unremployment rate change
full_df$last_unrate_chg_pos <- pmax(full_df$last_unrate_chg_yoy, 0)
#full_df <- full_df %>% select(-last_unrate_chg_yoy)

# Rate cut (negative part of change in fed funds rate)
full_df$last_rate_cut <- pmax(-full_df$last_fedfunds_chg_yoy, 0)
#full_df <- full_df %>% select(-last_fedfunds_chg_yoy)

# Indicator of change in yield curve slope being positive
full_df$last_slope_incr <- factor(as.integer(full_df$last_slope_chg_yoy > 0))
#full_df <- full_df %>% select(-last_slope_chg_yoy)

# Magnitude of slope (steepness)
```

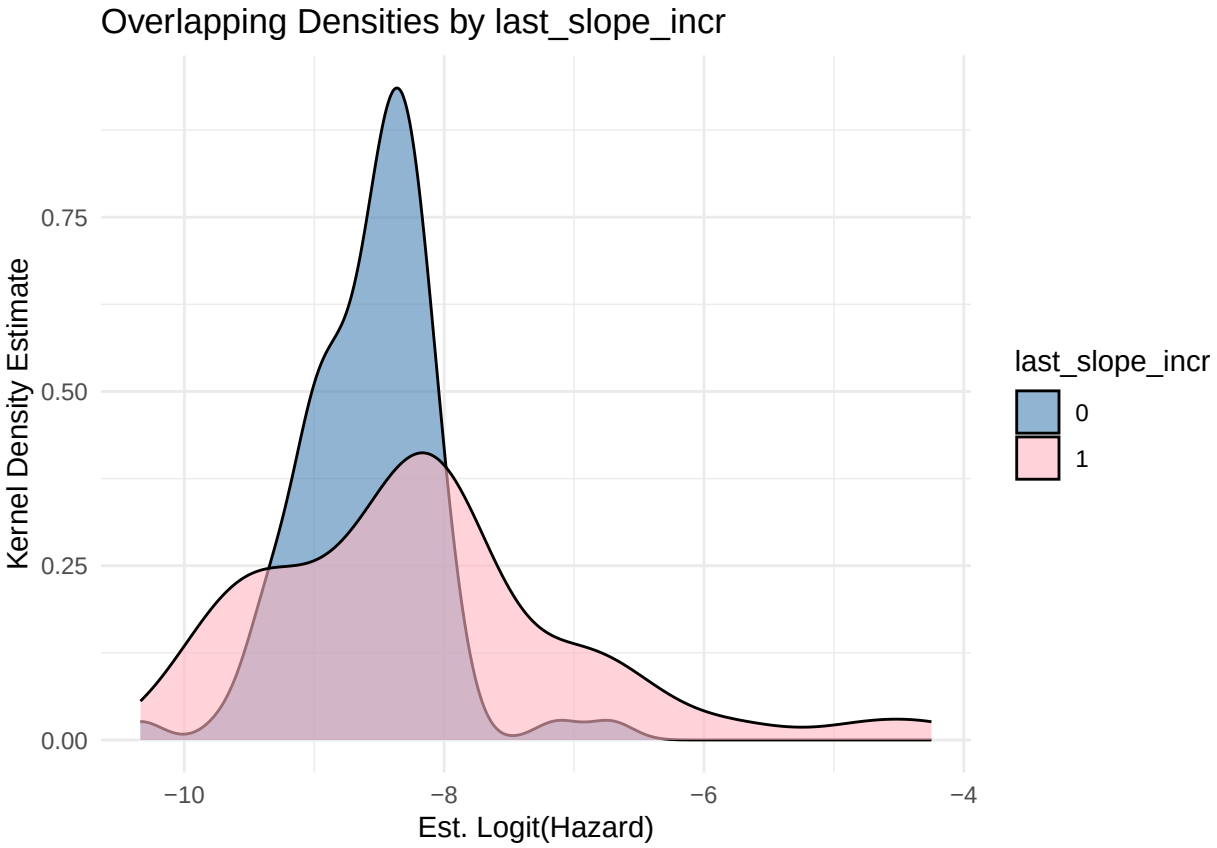
```r
full_df$last_slope_mag <- abs(full_df$last_slope)

# Inversion status of yield curve slope
full_df$last_invert <- factor(as.integer(full_df$last_slope < 0))
#full_df <- full_df %>% select(-last_slope)

# Binned VIX regimes
full_df$last_vix_bin <- factor(cut(full_df$last_vix, breaks=c(0,20,30,Inf)),
                               labels=c("Low", "Mid", "High"))
#full_df <- full_df %>% select(-last_vix)


new_vars <- c("last_unrate_chg_pos", "last_rate_cut", "last_slope_mag",
              "last_slope_incr", "last_infl_yoy_low", "last_invert",
              "last_vix_bin")

haz_date <- full_df %>%
  group_by(date) %>%
  summarise(haz = mean(y),across(all_of(new_vars), ~ first(.x)),
            .groups = "drop")

for (var_name in new_vars) {
  if (var_name %in% c("last_infl_yoy_low", "last_slope_incr",
                      "last_invert", "last_vix_bin")) {
    # overlapped histograms for categorical variables
    p <- ggplot(haz_date %>% filter(haz > 0 & haz < 1),
                aes(x = qlogis(haz), fill = factor(.data[[var_name]]))) +
      geom_density(alpha = 0.6, position = "identity") +
      scale_fill_manual(values = c("steelblue", "lightpink", "darkred"),
                        name = var_name, labels = c("0", "1", "2")) +
      theme_minimal() +
      labs(
        x = "Est. Logit(Hazard)",
        y = "Kernel Density Estimate",
        title = paste("Overlapping Densities by", var_name)
      )
  } else {
    smoother <- ifelse(test=(var_name=="last_unrate_chg_pos"),
                       yes="lm", no="loess")
    # scatterplots for continuous variables
    p <- ggplot(haz_date %>% filter(haz > 0 & haz < 1),
                aes(.data[[var_name]], qlogis(haz))) +
        geom_point() +
        geom_smooth(se = TRUE, method = smoother, formula = "y~x") +
        theme_minimal() +
        labs(y = "Est. Logit(Hazard)", x = var_name,
             title = var_name)
```

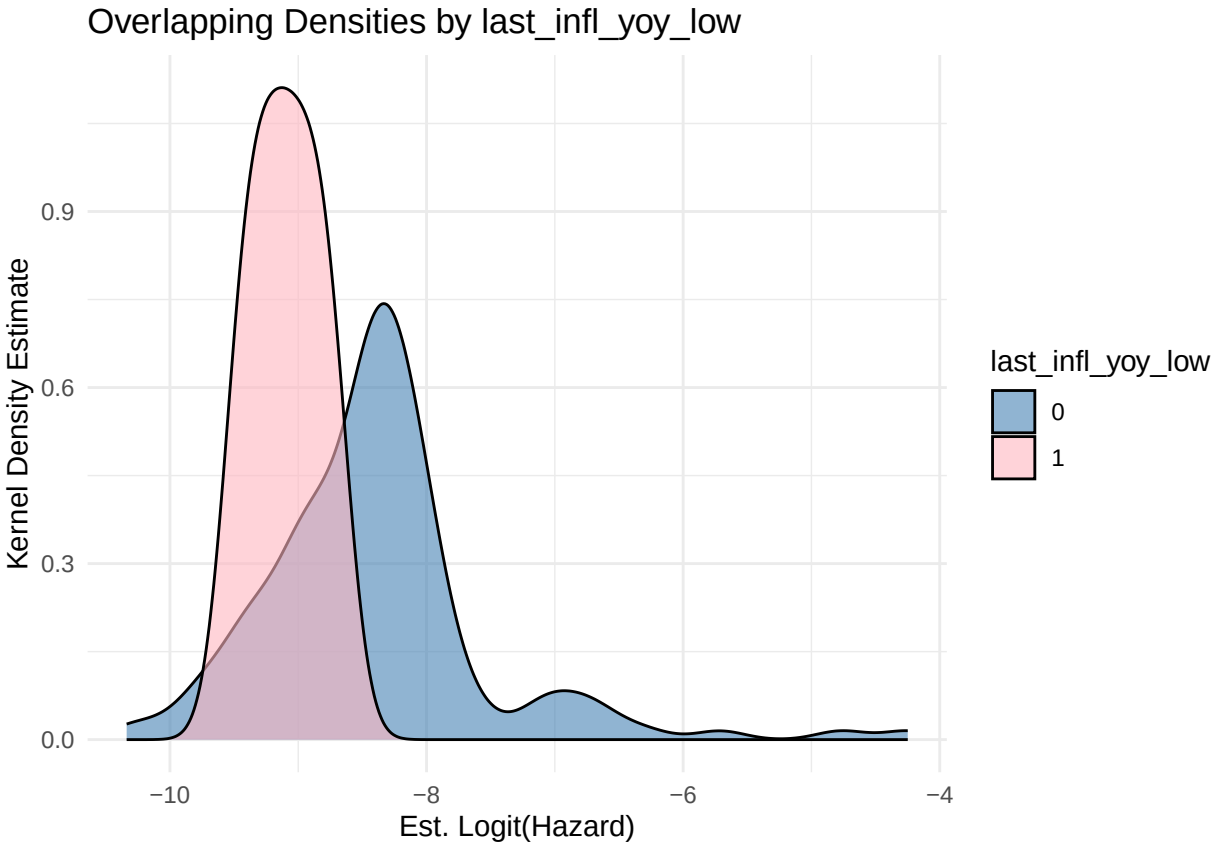
```
  }
  print(p)
}
```

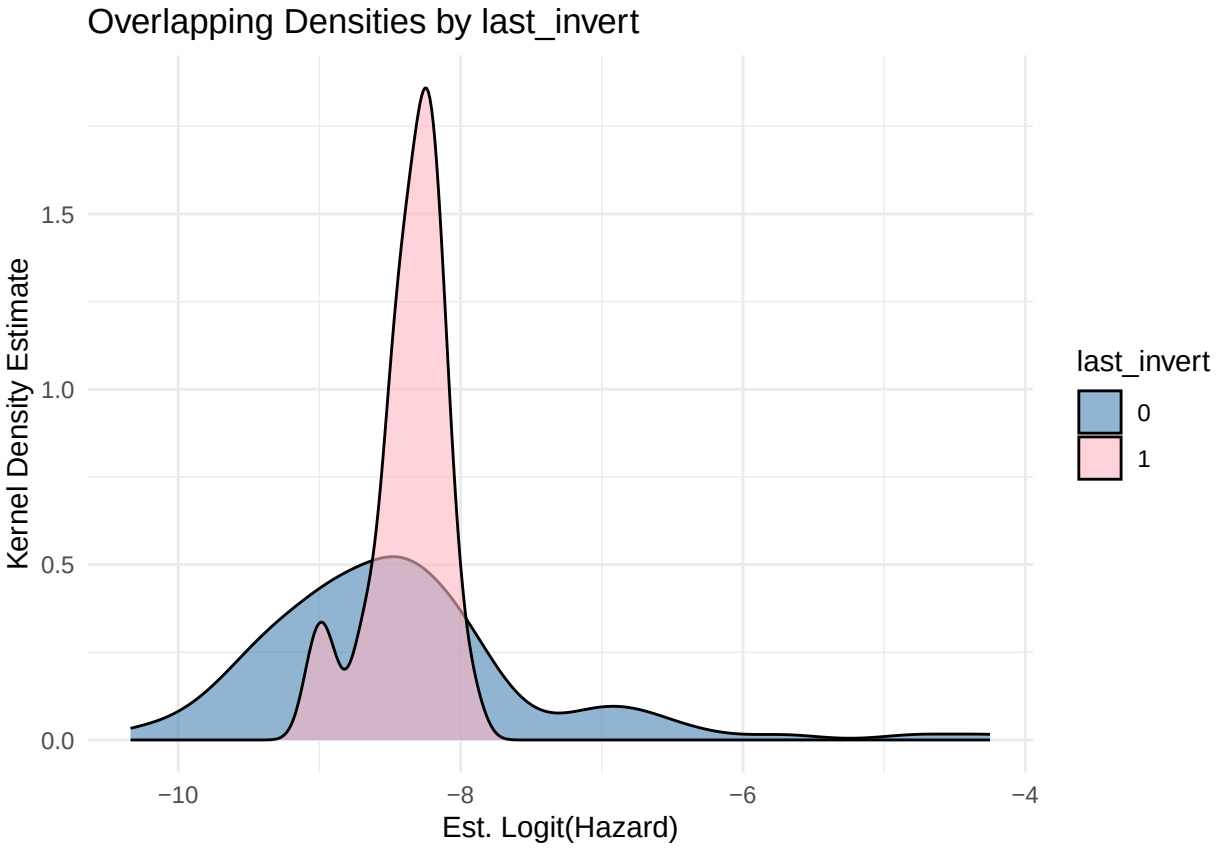

last_unrate_chg_pos
Est. Logit(Hazard)
−5
−7
−9
0
3
6
9
last_unrate_chg_pos

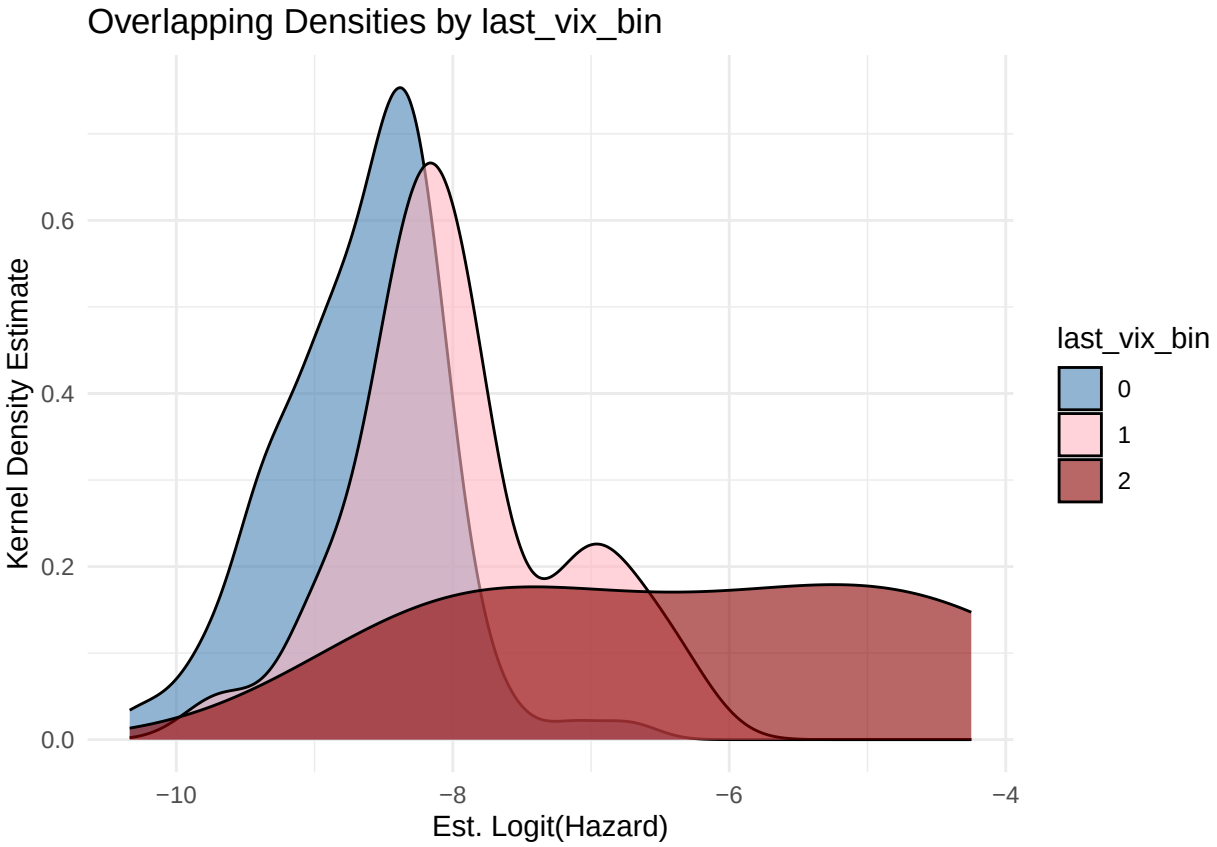

last_rate_cut
Est. Logit(Hazard)
−4
−6
−8
−10
0.0
0.5
1.0
1.5
2.0
last_rate_cut


last_slope_mag
Est. Logit(Hazard)
−4
−6
−8
−10
0
1
2
last_slope_mag


Overlapping Densities by last_slope_incr
Kernel Density Estimate
0.75
0.50
0.25
0.00
−10
−8
−6
−4
Est. Logit(Hazard)
last_slope_incr
0
1


Overlapping Densities by last_infl_yoy_low
Kernel Density Estimate
0.9
0.6
0.3
0.0
−10
−8
−6
−4
Est. Logit(Hazard)
last_infl_yoy_low
0
1


Overlapping Densities by last_invert
Kernel Density Estimate
1.5
1.0
0.5
0.0
−10
−8
−6
−4
Est. Logit(Hazard)
last_invert
0
1


Overlapping Densities by last_vix_bin
Kernel Density Estimate
0.6
0.4
0.2
0.0
−10
−8
−6
−4
Est. Logit(Hazard)
last_vix_bin
0
1
2

```
rm(haz_date)
```

`last_unrate_chg_pos` has a strong positive linear association with logit hazard.

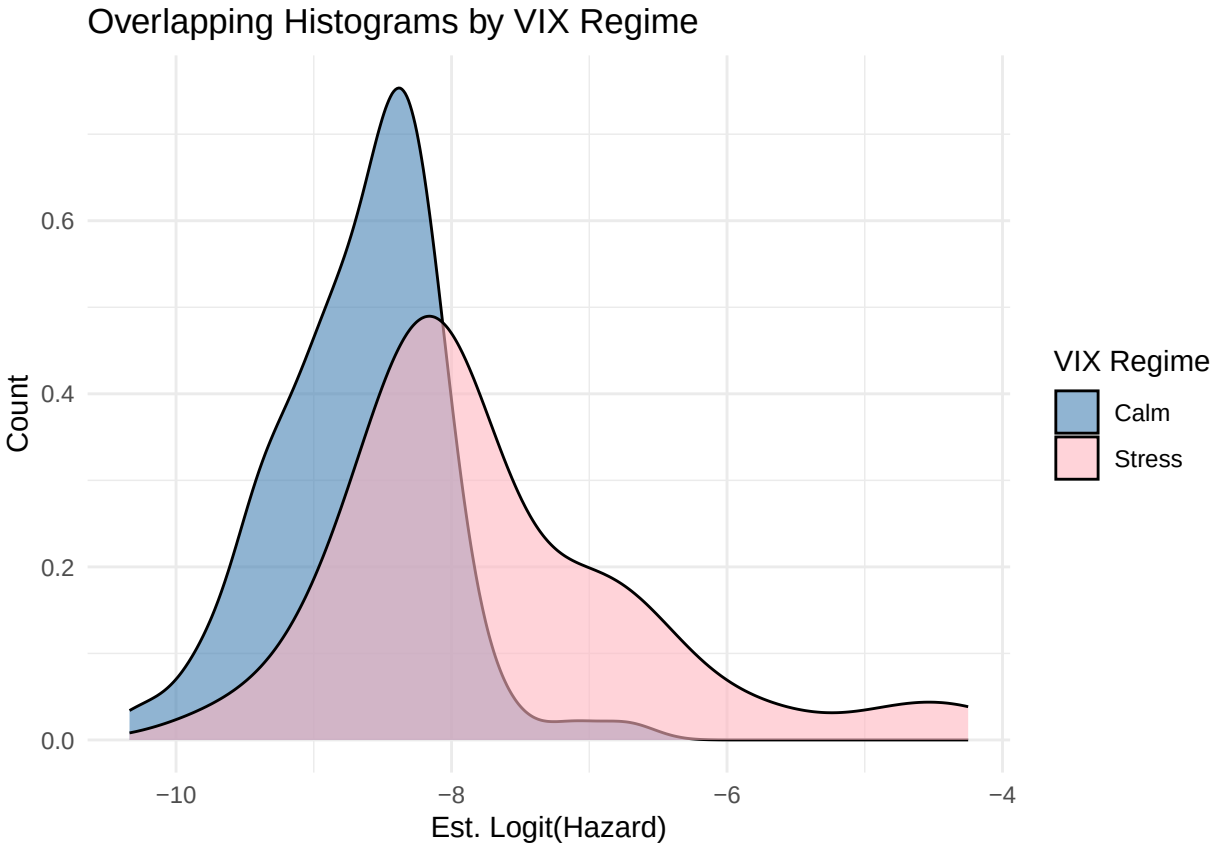
`last_rate_cut` has a moderate-strong rooughly linear association with logit hazard.

The distribution for the empirical logit hazard when `last_infl_yoy_low==1` (last month's YoY inflation was below 1.5%) is shifted left with lower variance, and it is shifted right with higher variance when `last_infl_yoy_low==0`, which suggests a material change in distribution, so we use this indicator in the model.

The kernel density estimates for the subset where `last_slope_incr==0` and `last_slope_incr==1` are centered at roughly the same logit hazard value so we do not use `last_slope_incr` as a predictor For the same reason we do not use `last_invert`.

Since there are very few data points with VIX > 30, we combine the two upper buckets into the indicator for VIX > 20 so that there is less class imbalance, which should help ensure coefficient stability.

```
full_df <- full_df %>% select(-c(last_slope_incr, last_invert))

# Binned VIX regimes (only 2 now)
full_df$last_vix_bin <- factor(cut(full_df$last_vix, breaks=c(0,20,Inf)),
                               labels=c("Calm", "Stress"))

haz_date <- full_df %>%
  group_by(date) %>%
  summarise(haz = mean(y), last_vix_bin = first(last_vix_bin),
            .groups = "drop")

ggplot(haz_date %>% filter(haz > 0 & haz < 1),
                aes(x = qlogis(haz), fill = factor(last_vix_bin))) +
      geom_density(alpha = 0.6, position = "identity") +
      scale_fill_manual(values = c("steelblue", "lightpink"),
                        name = "VIX Regime", labels = c("Calm", "Stress")) +
      theme_minimal() +
      labs(
        x = "Est. Logit(Hazard)",
        y = "Count",
        title = paste("Overlapping Histograms by", "VIX Regime")
      )
```



The kernel density estimates show that the distribution is shifted right for `last_vix_bin==1` (stress regime) with higher variance, while the distribution for `last_vix_bin==0` (calm regime) is more tightly centered (lower variance) around a lower mode.

Looking at Approximate LTV

Earlier in the code, we derived the time-varying approximate Home Price Appreciation since Origination (HPAO) variable as

$$\text{HPAO}_i(t) = \text{HPI}(t) \Big/ \text{HPI}(t_0^{(i)})$$

where HPI_t is the Home Price Index at time t and $t = t_0^{(i)}$ corresponds to the time of origination for loan i. We mentioned earlier that the LTV (Loan-To-Value) ratio for a given loan i at time $t > t_0^{(i)}$ is estimated by Freddie Mac using their Automated Valuation Model, which is because the value of the home for loan i is typically unknown at an arbitrary time after origination. Using the $\text{HPAO}_i(t)$ variable defined above, we estimate $\text{LTV}_i(t)$ as follows.

Let L_i be the original loan amount for loan i and let $V_i(t)$ be the value[4]

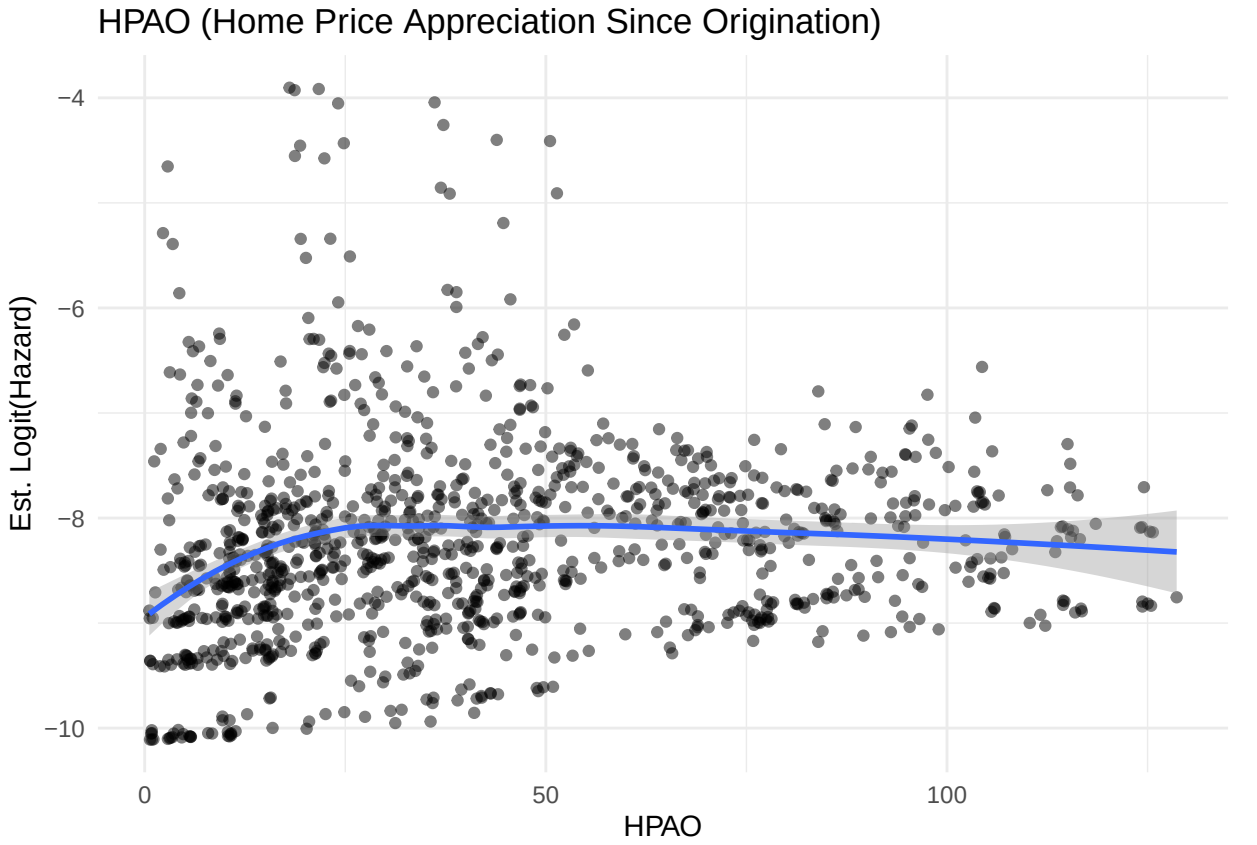
$$\text{LTV}_i(t) = \frac{\text{L}_i}{V_i(t)} = \frac{\text{L}_i}{V_i(t) \cdot V_i(t_0^{(i)}) / V_i(t_0^{(i)})} = \frac{\text{L}_i}{V_i(t_0^{(i)})} \cdot \frac{1}{V_i(t)/V_i(t_0^{(i)})} \approx \text{LTV}_i(t_0^{(i)}) \Big/ \text{HPAO}_i(t)$$

[4]When we say "value" here, we really just mean the market price, not necessarily the intrinsic value or fair value in a traditional valuation sense.

We use the assumption that the growth in the value of home i is approximately the same as the growth in HPI.

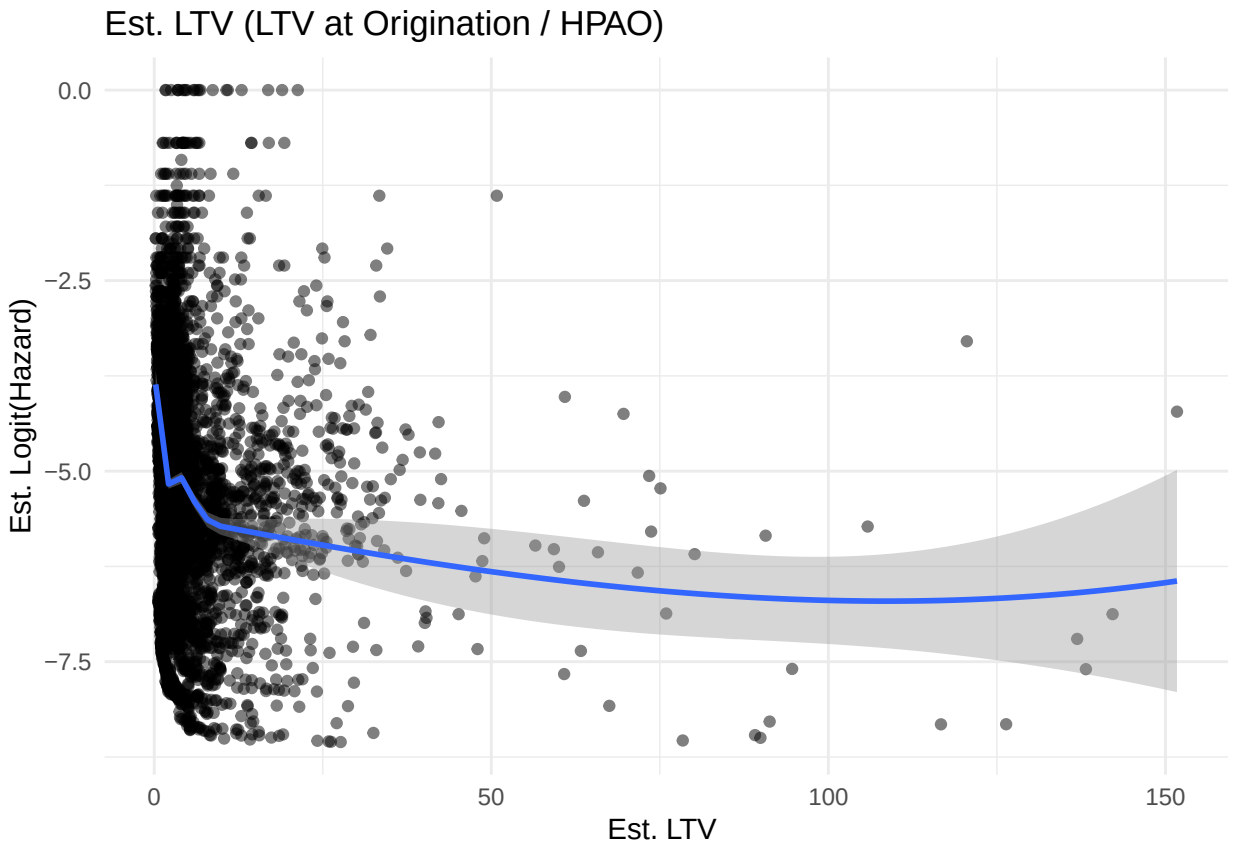
Now we inspect the possible use of the 1-month lagged versions of $\text{HPAO}_i(t)$ and/or $\text{LTV}_i(t_0^{(i)})\big/\text{HPAO}_i(t)$ as covariates in the model.

```
haz_hpao <- full_df %>%
  group_by(last_hpao) %>%
  summarise(haz = mean(y), .groups = "drop")
ggplot(haz_hpao %>% filter(haz > 0 & haz < 1),
       aes(x = last_hpao, y = qlogis(haz))) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "loess", se = TRUE, formula = "y ~ x") +
  theme_minimal() +
  labs(
    x = "HPAO",
    y = "Est. Logit(Hazard)",
    title = "HPAO (Home Price Appreciation Since Origination)"
  )
```



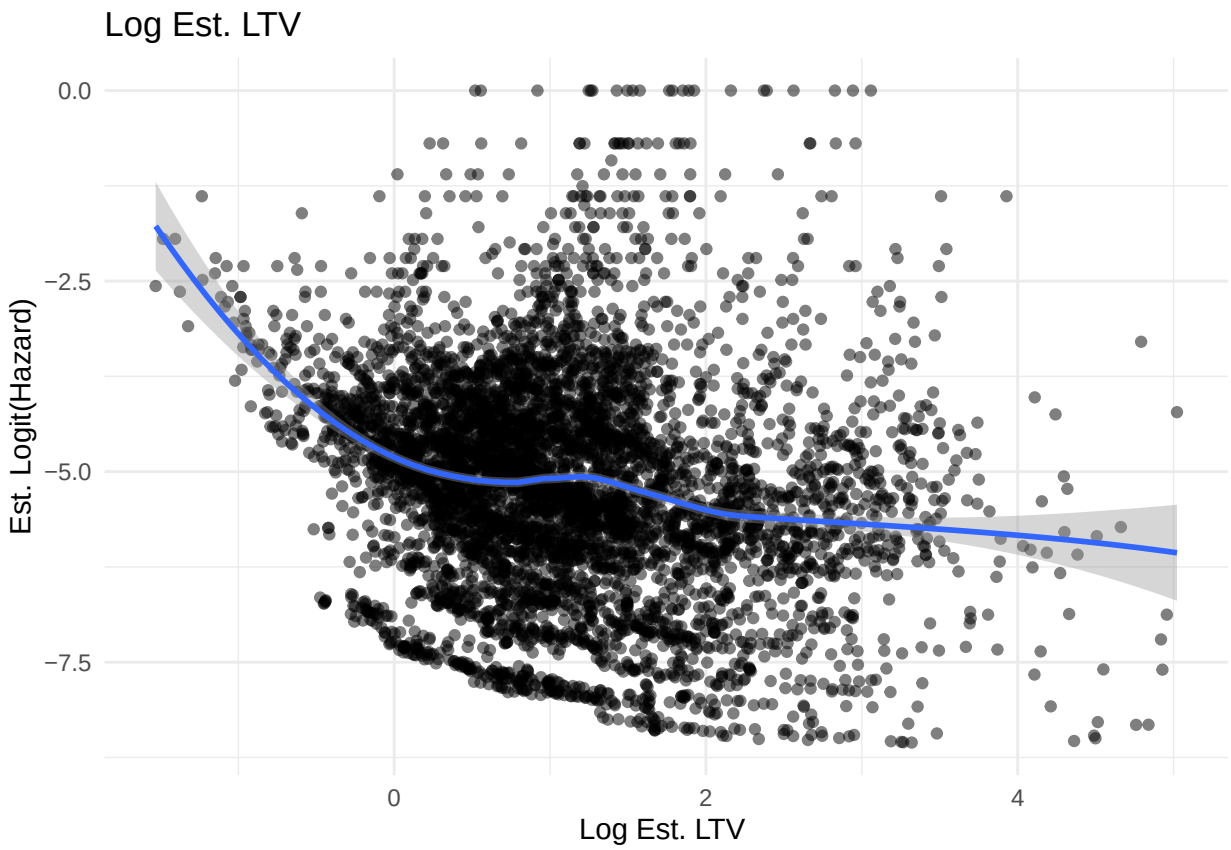

```
rm(haz_hpao)


full_df <- full_df %>%
  mutate(last_ltv_est = orig_ltv / last_hpao)
```

```r
haz_eltv <- full_df %>%
  group_by(last_ltv_est) %>%
  summarise(haz = mean(y), .groups = "drop")
ggplot(haz_eltv %>% filter(haz > 0 & haz < 1),
       aes(x = last_ltv_est, y = qlogis(haz))) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "loess", se = TRUE, formula = "y ~ x") +
  theme_minimal() +
  labs(
    x = "Est. LTV ",
    y = "Est. Logit(Hazard)",
    title = "Est. LTV (LTV at Origination / HPAO)"
  )
```

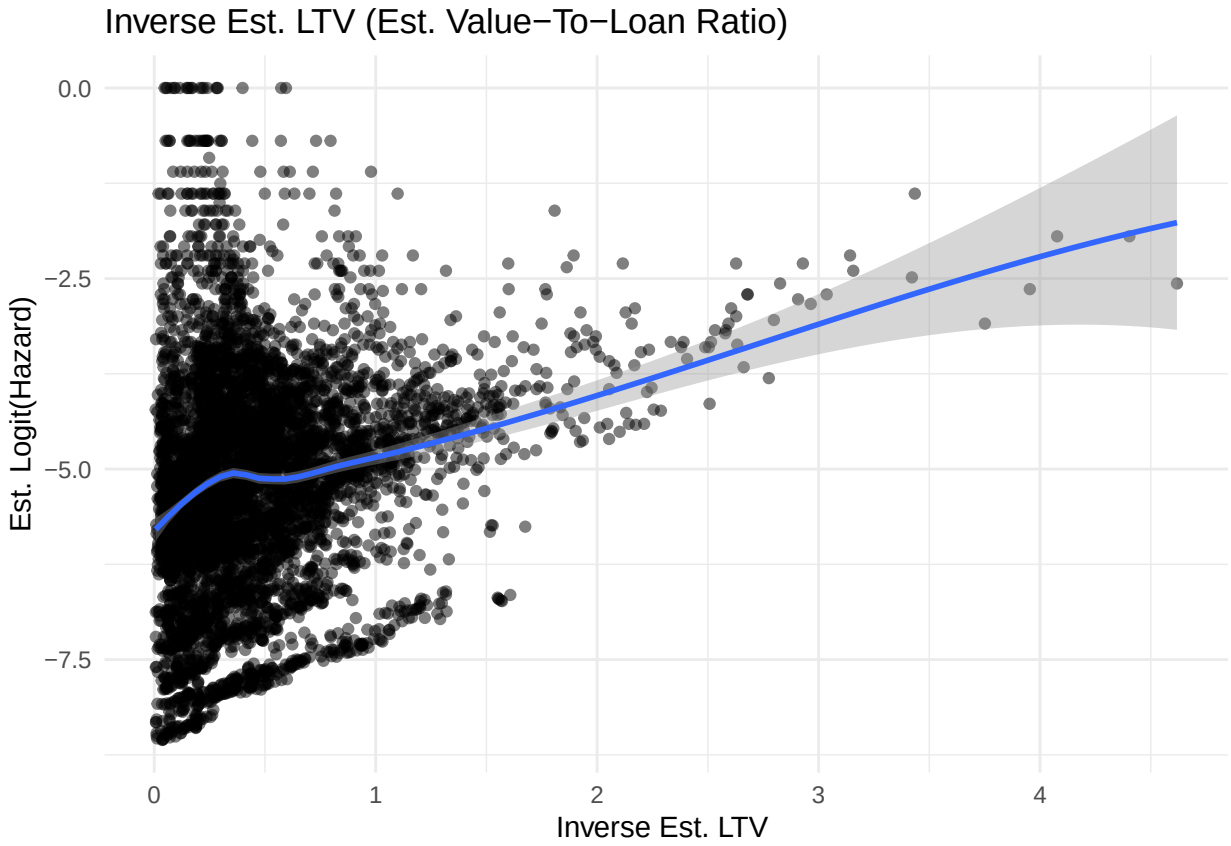



```r
ggplot(haz_eltv %>% filter(haz > 0 & haz < 1),
       aes(x = log(last_ltv_est), y = qlogis(haz))) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "loess", se = TRUE, formula = "y ~ x") +
  theme_minimal() +
  labs(
    x = "Log Est. LTV ",
    y = "Est. Logit(Hazard)",
    title = "Log Est. LTV"
```

```
  )
```




```
ggplot(haz_eltv %>% filter(haz > 0 & haz < 1),
       aes(x = 1/last_ltv_est, y = qlogis(haz))) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "loess", se = TRUE, formula = "y ~ x") +
  theme_minimal() +
  labs(
    x = "Inverse Est. LTV ",
    y = "Est. Logit(Hazard)",
    title = "Inverse Est. LTV (Est. Value-To-Loan Ratio)"
  )
```




```
rm(haz_eltv)
```

The raw estimated LTV covariate shows a weak nonlinear relationship with logit hazard, and the log transform of the covariate shows a stronger but still somewhat nonlinear relationship with logit haazard. The inverse transform (which is effectively the estimated Value-To-Loan ratio) shows a moderate-strong linear relationship, so this is what we use in the model.

The Value-To-Loan ratio can be viewed as a dimensionless version of the equity of the home:

Let E be the equity of the home, V be the value of the home and L be the loan amount, all at some fixed time. Now we have the following sequence of bijections: $E = V - L \mapsto \frac{V-L}{L} = V/L - 1 \mapsto V/L$

This makes sense economically and aligns with the structural (option-theoretic) view of mortgages, wherein the mortgage is essentially viewed as put option on the home with the strike being the loan amount.

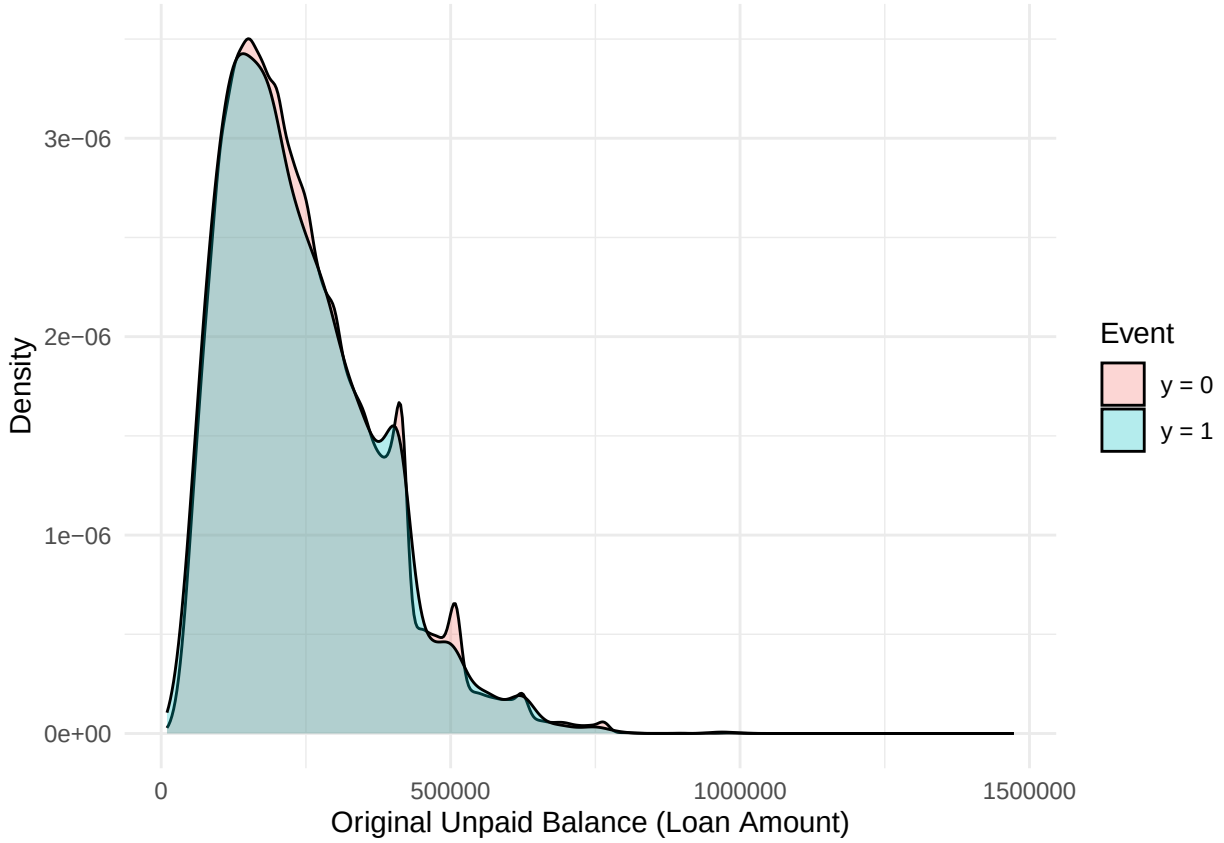
- (Negative Equity Case) $V < L \implies$ default (exercising the put) is economically rational.
- (Positive Equity Case) $V > L \implies$ the put is out of the money and default (exercise) probability is small.

possible source: Deng, Y., Quigley, J., & Van Order, R. (2000). Mortgage Terminations, Heterogeneity and the Exercise of Mortgage Options. Econometrica.

Analyze Rest of Design Matrix and Assess Multicollinearity

Let's look at the original loan amount as a potential predictor.

```r
origupb_plot_df <- full_df %>%
  group_by(loan_sequence_number) %>%
  summarise(amt = min(orig_upb), event=sum(y)) %>%
  ungroup() %>%
  mutate(event = factor(event, levels=c(0,1), labels=c("y = 0", "y = 1")))

ggplot(origupb_plot_df, aes(x=amt, fill=event)) +
  geom_density(position='identity', alpha=0.3) +
  labs(x="Original Unpaid Balance (Loan Amount)", y="Density", fill="Event") +
  theme_minimal()
```




```r
rm(origupb_plot_df)
```

The two kernel density estimates are nearly identical so it is clear that there does not seem to be any predictive power in the loan amount variable.

We now look at sample summaries of the outcome across different levels of our categorical predictors to ensure that the event counts are sufficiently balanced across all levels.

```r
# Remove variables already deemed to have no predictive power
full_df <- full_df %>%
  select(-loan_age, -loan_age_pct, -orig_upb)

full_df$y <- full_df$y %>% as.integer()
```

```r
# Encode categorical variables as factors
cat_vars <- c("deliq_num", "occupancy_status", "region", "loan_purpose")
for (v in cat_vars[1:length(cat_vars)]) {
  full_df[[v]] <- as.factor(full_df[[v]])
}

# Summarize response grouped by each categorical variable
results <- lapply(cat_vars, function(v) {
  full_df %>% group_by(.data[[v]]) %>%
    summarise(mu = mean(y), sd = sqrt(mean(y)*(1-mean(y))),
              n  = n(), .groups = "drop") %>%
    mutate(variable = v) %>%
    rename(level = 1) %>%
    select(variable, level, everything())
})
catsum_df <- bind_rows(results) %>%
  group_by(variable) %>%
  arrange(variable, mu) %>%
  ungroup()
catsum_df %>% kable()
```

variable	level	mu	sd	n
deliq_num	0	0.0000038	0.0019394	18345658
deliq_num	1	0.0006137	0.0247647	94514
deliq_num	2	0.4630041	0.4986294	19786
loan_purpose	N	0.0004298	0.0207264	8809379
loan_purpose	C	0.0005139	0.0226627	3487319
loan_purpose	P	0.0006020	0.0245274	6163260
occupancy_status	S	0.0003404	0.0184466	693317
occupancy_status	P	0.0005075	0.0225218	16274228
occupancy_status	I	0.0005314	0.0230450	1492413
region	North Central	0.0003879	0.0196923	4420807
region	West	0.0004518	0.0212517	5276223
region	South	0.0005917	0.0243169	5991598
region	Northeast	0.0005932	0.0243488	2771330

For `deliq_num` (the number of monthly payments missed), we see that the estimated PD (`mu`) becomes orders of magnitude higher as the level increases (strong association), so we include it in the model. In fact, the inclusion of this covariate is what effectively makes our model an incomplete Markov transition model in the sense that it would be complete if we also had models for the probability of `deliq_num=2` and `deliq_num=1` (which is attainable, just out of scope for this project). Note that we could also remove this covarite to make our model more of a structural economic model rather than a state-transition model. *verify that details of this are sound*

We still have class imbalance, as expected.

For each of the other categorical variables, the estimated PD is the same order of magnitude (10^{-4})

across all levels.

For `loan_purpose`, we see that the estimated PD is descending in the order `(P,C,N)`, where P corresponds to purchase borrowers, C corresponds to cash-out refinance (ReFi) borrowers, and N corresponds to no-cash-out ReFi borrowers[5]. The relative growth values in estimated PD as we move across these levels is large, so we include `loan_purpose` in the model.

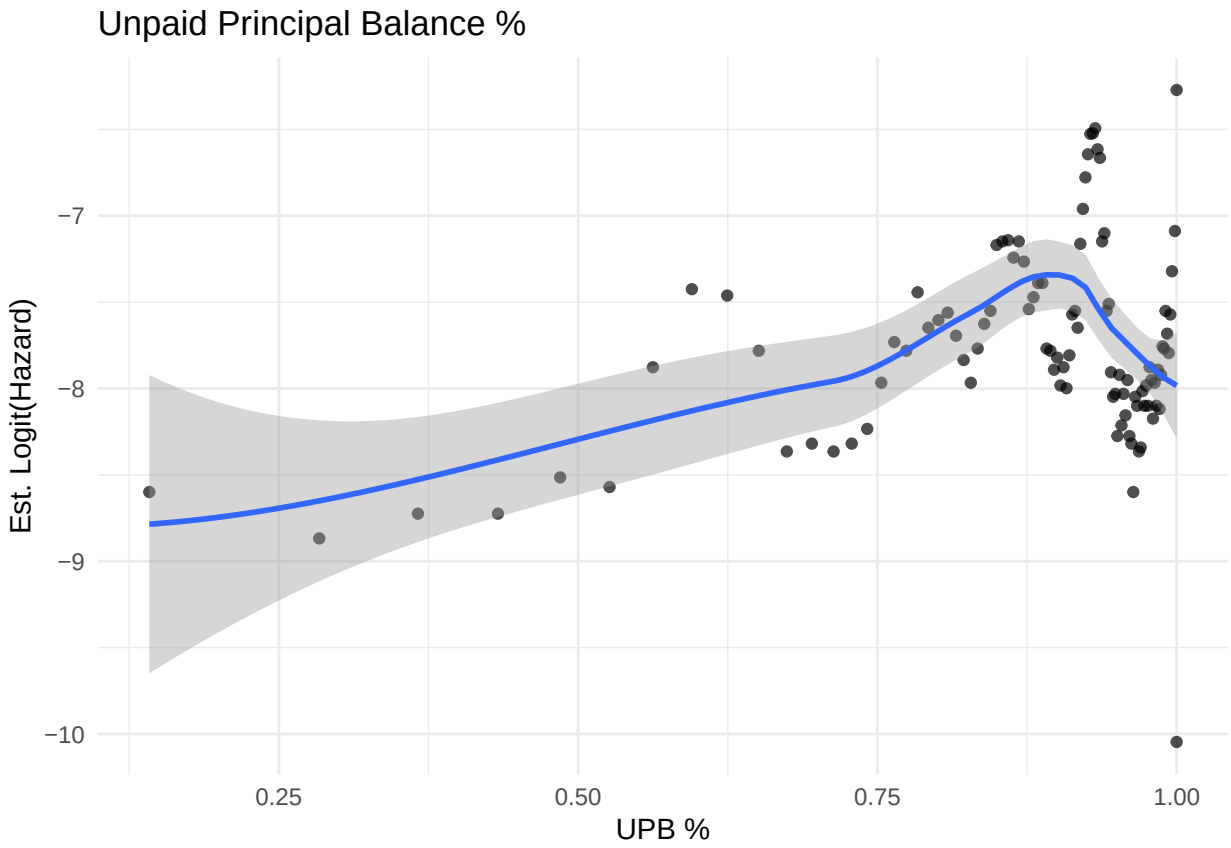
For `occupancy_status`, we see that loans for secondary homes (`S`) have the lowest est. PD, loans for primary residences (`P`) have an est. PD that is relatively higher, and investment properties (`I`) have the highest est. PD although it is only a little higher than that of `P`. The effect of occupancy status seems weak overall, so we may include it as a weak but interepretable control although it does not seem to be a major risk factor.

For `region`, we observe differences in est. PD across the levels on the same relative scale as seen in the `occupancy_status` factor, so it does not seem to be a major risk factor although it may be useful to include in the model as a control, especially since the effects of economic shocks can vary with respect to regional geopolitical status.

Now let's look at potential continuous predictors

```
# do FOR loop similar to for macro variables except will have to recompute haz df each time


haz_upb_bin <- full_df %>%
  mutate(upb_bin = ntile(upb_pct, 100)) %>%
  group_by(upb_bin) %>%
  summarise(upb_mu = mean(upb_pct), haz = mean(y), .groups = 'drop')
ggplot(haz_upb_bin %>% filter(haz > 0 & haz < 1),
       aes(upb_mu, qlogis(haz))) +
  geom_point(alpha=0.7) +
  geom_smooth(se = TRUE, method = "loess", formula = "y~x") +
  theme_minimal() +
  labs(y = "Est. Logit(Hazard)", x = "UPB %",
       title = "Unpaid Principal Balance %")
```

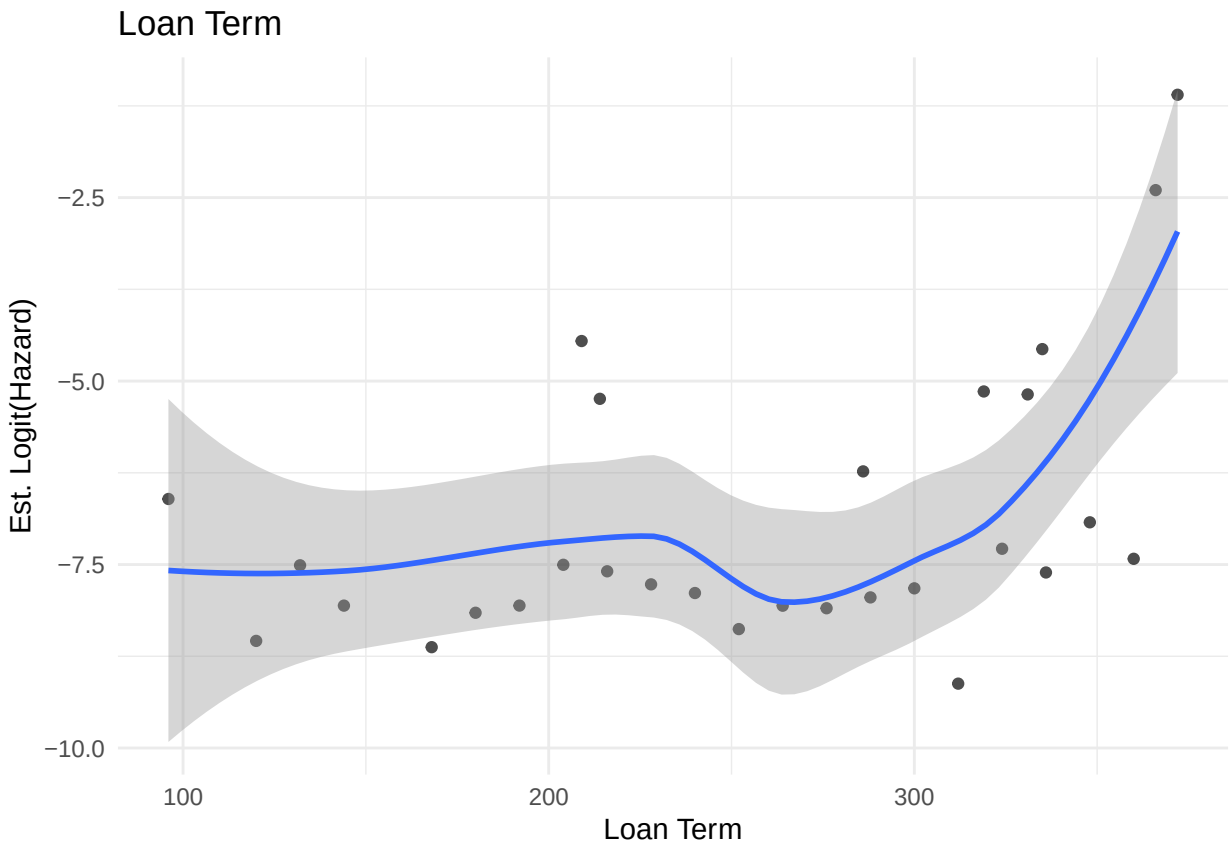
[5]Details on what these terms mean can be found on https://www.freddiemac.com/fmac-resources/research/pdf/user_guide.pdf under the "Origination Data File" table in column position 21.



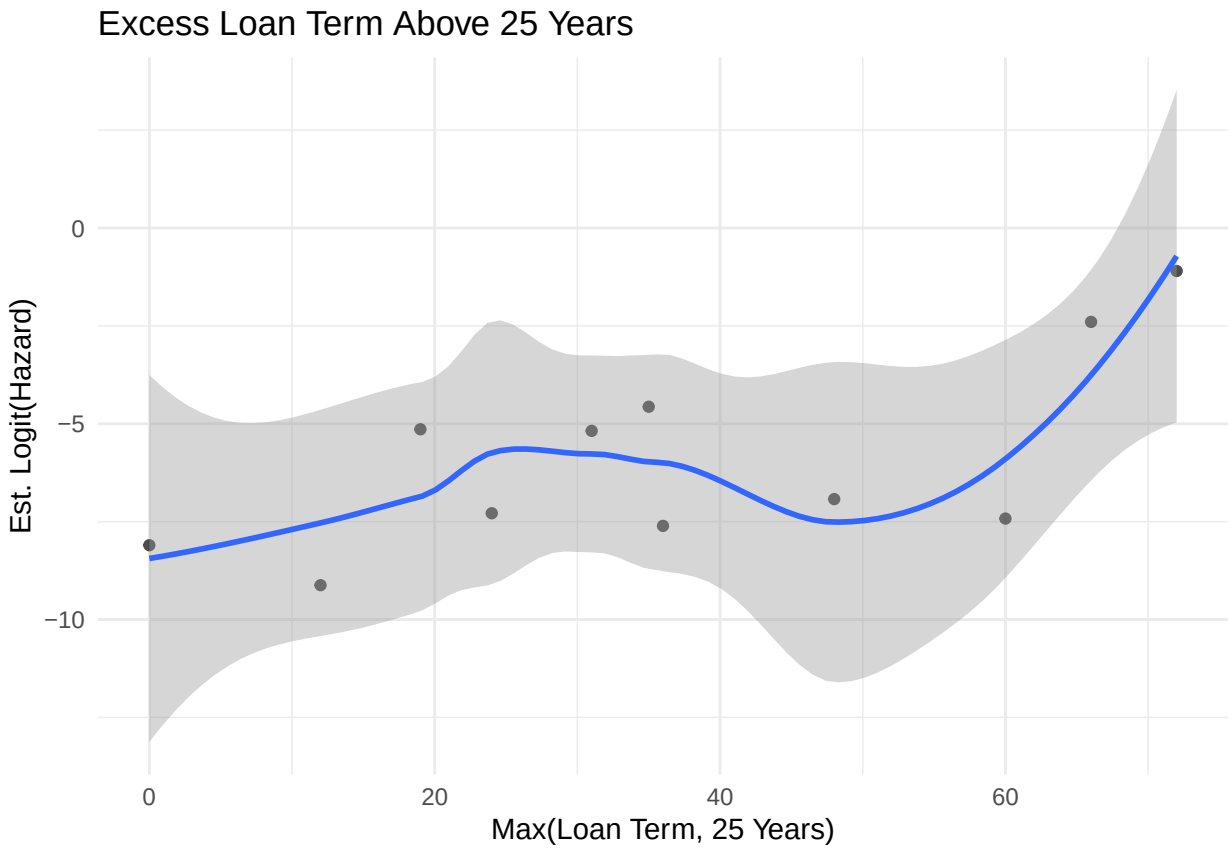

```
rm(haz_upb_bin)
```

The unpaid principal balance % covariate (`upb_pct`) is strongly collinear with loan age and is a deterministic function of the payment schedule conditional on the absence of missed payments. *need to further justify not including it since yeah vs. logit hazard it has weird wave behaviour but it also is overall pretty linear and strong association*

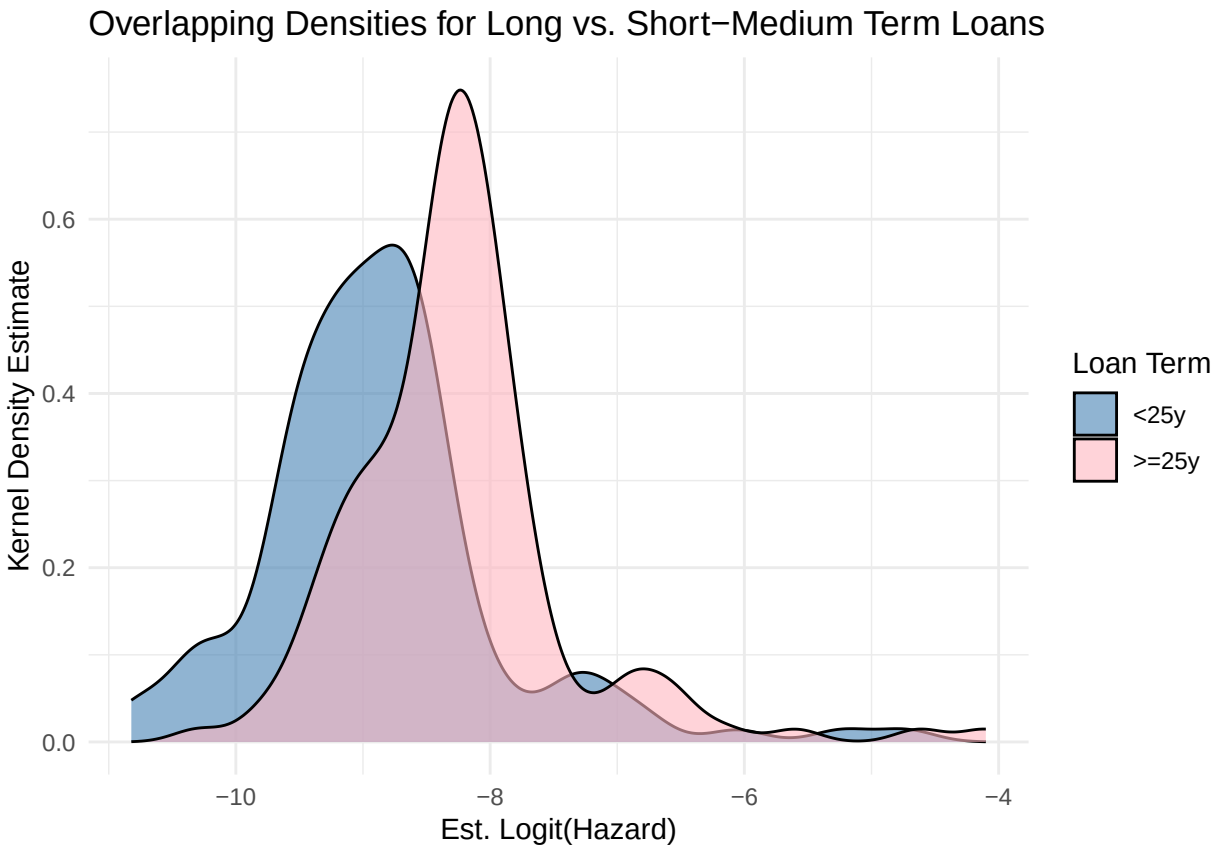
```
# Looking at loan term
haz_term <- full_df %>%
  group_by(orig_loan_term) %>%
  summarise(haz = mean(y), .groups = 'drop')
ggplot(haz_term %>% filter(haz > 0 & haz < 1),
       aes(orig_loan_term, qlogis(haz))) +
  geom_point(alpha=0.7) +
  geom_smooth(se = TRUE, method = "loess", formula = "y~x") +
  theme_minimal() +
  labs(y = "Est. Logit(Hazard)", x = "Loan Term",
       title = "Loan Term")
```




```
rm(haz_term)

# Excess loan term above 25yrs
full_df$term_cap <- pmax(full_df$orig_loan_term - 300, 0)
haz_term <- full_df %>%
  group_by(term_cap) %>%
  summarise(haz = mean(y), .groups = 'drop')
ggplot(haz_term %>% filter(haz > 0 & haz < 1),
       aes(term_cap, qlogis(haz))) +
  geom_point(alpha=0.7) +
  geom_smooth(se = TRUE, method = "loess", formula = "y~x") +
  theme_minimal() +
  labs(y = "Est. Logit(Hazard)", x = "Max(Loan Term, 25 Years)",
       title = "Excess Loan Term Above 25 Years")
```



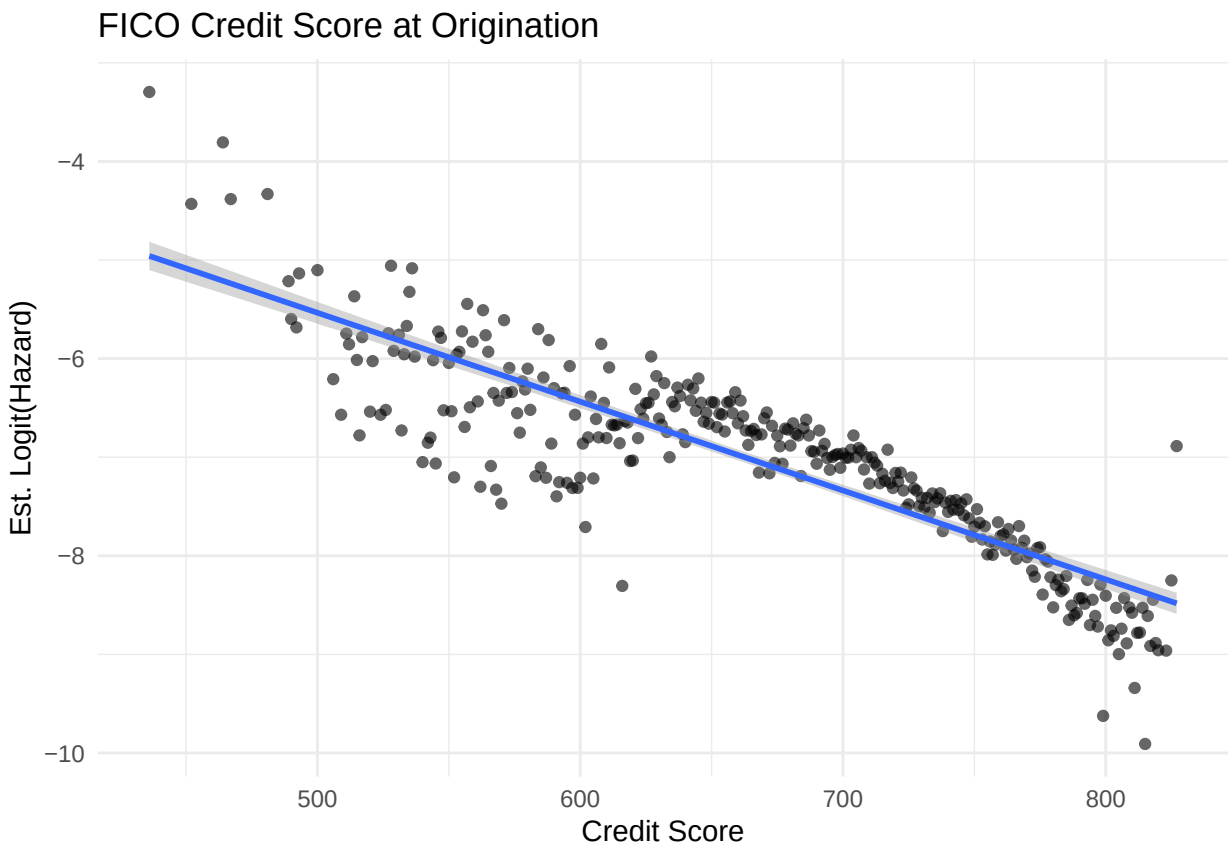

```
rm(haz_term)
full_df <- full_df %>% select(-term_cap)


# Indicator for loan term being at least 25 years
full_df$long_term <- factor(as.integer(full_df$orig_loan_term >= 300),
                            labels=c("<25y", ">=25y"))
haz_term <- full_df %>%
  group_by(date, long_term) %>%
  summarise(haz = mean(y), .groups = "drop")
ggplot(haz_term %>% filter(haz > 0 & haz < 1),
       aes(x = qlogis(haz), fill = long_term)) +
  geom_density(alpha = 0.6, position = "identity") +
  scale_fill_manual(
    values = c("steelblue", "lightpink"),
    name   = "Loan Term",
    labels = c("<25y", ">=25y")
  ) +
  theme_minimal() +
  labs(x = "Est. Logit(Hazard)", y = "Kernel Density Estimate",
    title = "Overlapping Densities for Long vs. Short-Medium Term Loans"
  )
```




```
rm(haz_term)
```

comment about including I(loan_term >= 300) in the model but not the others

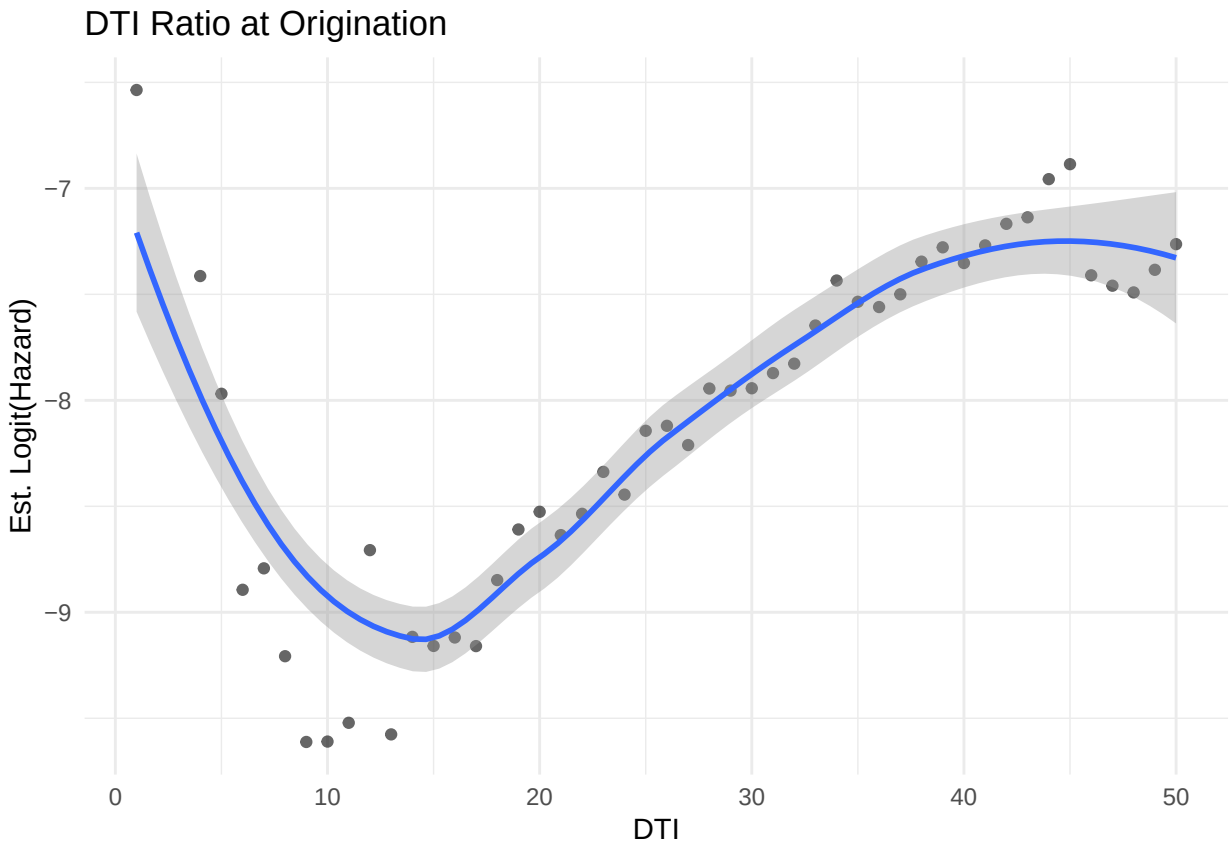
```
# Looking at credit score
haz_credit <- full_df %>%
  group_by(credit_score) %>%
  summarise(haz = mean(y), .groups = "drop")
ggplot(haz_credit %>% filter(haz > 0 & haz < 1),
       aes(y = qlogis(haz), x = credit_score)) +
  geom_point(alpha = 0.6, position = "identity") +
  geom_smooth(method='lm', se=TRUE, formula='y~x') +
  theme_minimal() +
  labs( x = "Credit Score", y = "Est. Logit(Hazard)",
    title = "FICO Credit Score at Origination"
  )
```



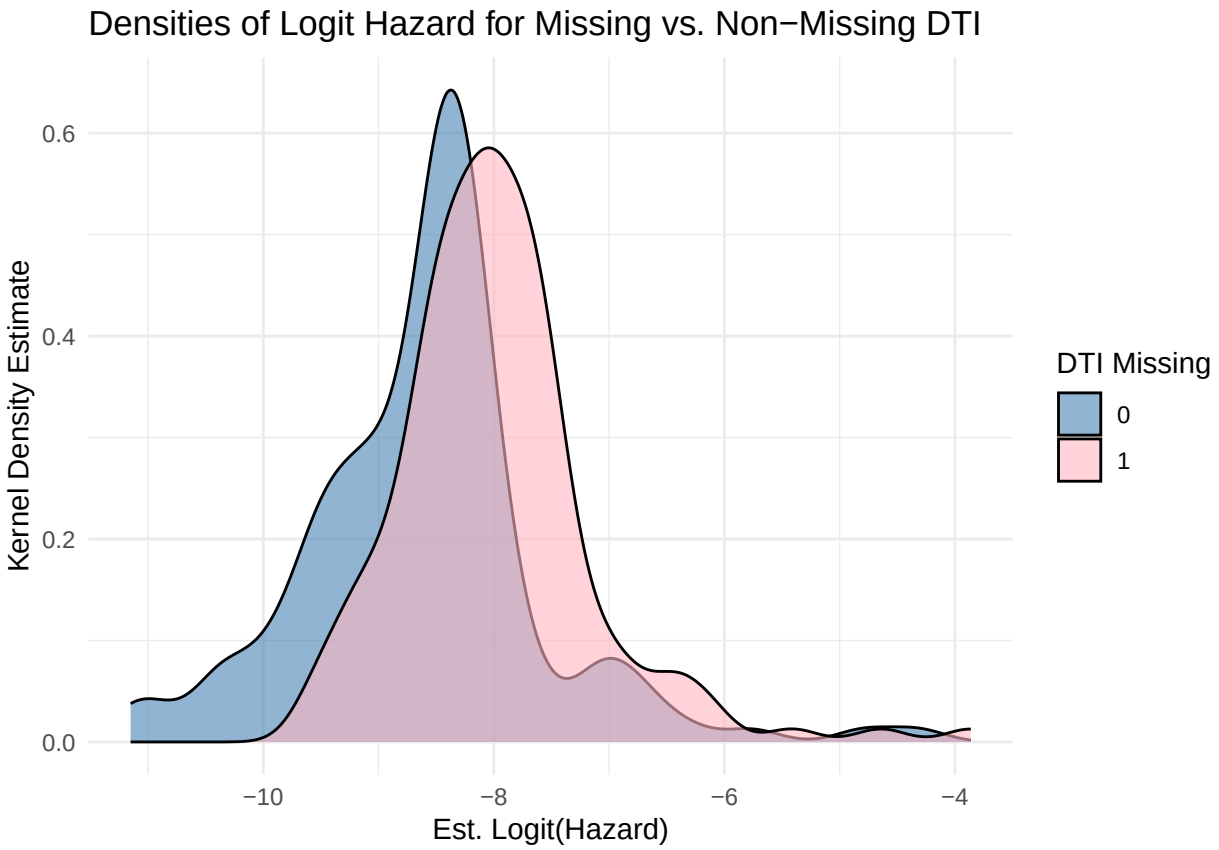

```
rm(haz_credit)
```

Credit score shows a strong negative linear association with logit hazard. From the context of our data, we know that FICO scores are typically used when determining the interest rate (at origination), and the interest rate is a function of credit spread and the 10-year treasury rate. Thus, it may seem redundant to include both the credit spread and FICO score at origination and the credit spread at origination, however the correlation is actually quite low in our sample, which we suspect is reflecting the fact that much more than *just* the credit score goes into loan pricing (determining the interest rate to charge). So we include both in the model. We will check VIF scores later to do a final check of multicollinearity.

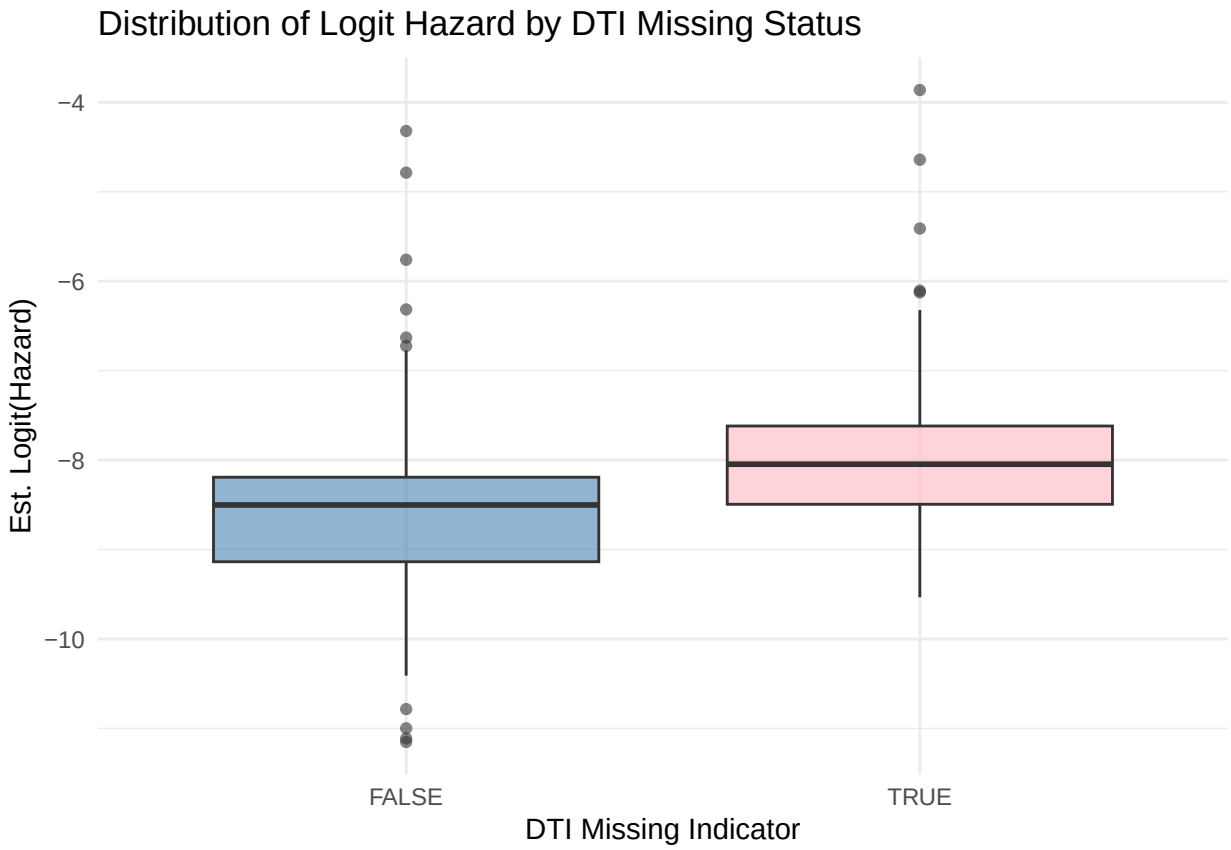
```
# Looking at DTI ratio at origination
haz_dti <- full_df %>%
  group_by(orig_dti_imp) %>%
  summarise(haz = mean(y), .groups = "drop")
ggplot(haz_dti %>% filter(haz > 0 & haz < 1),
       aes(y = qlogis(haz), x = orig_dti_imp)) +
  geom_point(alpha = 0.6, position = "identity") +
  geom_smooth(method='loess', se=TRUE, formula='y~x') +
  theme_minimal() +
  labs( x = "DTI", y = "Est. Logit(Hazard)",
    title = "DTI Ratio at Origination"
  )
```




```
rm(haz_dti)

haz_dti_miss <- full_df %>%
  group_by(dti_missing, date) %>%
  summarise(haz = mean(y), .groups = "drop")
ggplot(haz_dti_miss %>% filter(haz > 0 & haz < 1),
                aes(x = qlogis(haz), fill = factor(dti_missing))) +
      geom_density(alpha = 0.6, position = "identity") +
      scale_fill_manual(values = c("steelblue", "lightpink"),
                        name = "DTI Missing", labels = c("0", "1")) +
      theme_minimal() +
      labs(x = "Est. Logit(Hazard)", y = "Kernel Density Estimate",
        title = "Densities of Logit Hazard for Missing vs. Non-Missing DTI")
```




```
ggplot(haz_dti_miss %>% filter(haz > 0 & haz < 1),
       aes(x = factor(dti_missing), y = qlogis(haz),
           fill = factor(dti_missing))) +
  geom_boxplot(alpha = 0.6) +
  scale_fill_manual(values = c("steelblue", "lightpink"), name = "DTI Missing") +
  theme_minimal() +
  labs(
    x = "DTI Missing Indicator",
    y = "Est. Logit(Hazard)",
    title = "Distribution of Logit Hazard by DTI Missing Status"
  ) +
  theme(legend.position = "none")
```




```
rm(haz_dti_miss)

full_df %>%
  group_by(dti_missing) %>%
  summarise(haz = mean(y), sd = sqrt(haz*(1-haz)), n = n(),
            .groups = "drop") %>%
  kable()
```

dti_missing	haz	sd	n
FALSE	0.0004839	0.0219935	15662858
TRUE	0.0006106	0.0247034	2797100

We include `dti_missing` in the model due to the apparent shift in distribution that we observe across the groups.

The imputed DTI at origination variable clearly has a nonlinear relationship with logit hazard, so we try using splines vs. piecewise linear specifications for this term and we will compare the resulting models.

```
# can do GLM comparison via deviance/AIC right here with just DTI as predictor or do it later
```

We do not include the LTV ratio at origination since we are already including the approximate dynamic LTV as derived and analysed earlier.

now define the final vector of covariates and look at multicollinearity

next, do LASSO (with CV for hyperparam) for final var select and finally then do interpret + prediction

Next, we look at the correlation matrix of continuous variables to check for multicollinearity.

```
# Marginal distribution of each continuous variable
num_df <- full_df %>% select_if(is.numeric)
par(mfrow=c(3,3))
for (vec_name in names(num_df)) {
  boxplot(num_df[[vec_name]], main=vec_name, xlab=vec_name)
}
par(mfrow=c(1,1))


# Correlation matrix heat map for all continuous variables
cor_num <- cor(num_df)
corrplot(cor_num, method="color", tl.col="black", tl.cex=0.9,
         number.cex=0.4, addCoef.col="black", tl.srt=45, number.font=3,
         title="Correlations (Continuous Variables)")




# Check VIFs for all variables
fit_all <- glm(y ~ loan_age + I(loan_age^2) + deliq_num +
                 upb_pct + rate_change +
                 last_unrate + last_unrate_chg_yoy +
                 last_fedfunds + last_fedfunds_chg_yoy +
                 last_hpi_lg_yoy + last_infl_lg_yoy,
               family=binomial, data=full_df)

fit_all2 <- glm(
  event ~
    loan_age + I(loan_age^2) +
    last_unrate + last_unrate_chg_yoy +
    last_fedfunds + last_fedfunds_chg_yoy +
    last_hpi_lg_yoy + last_infl_lg_yoy,
  family = binomial,
  data = tv_surv_df
)


check_collinearity(fit_adj)
```

Comment on correlations observed in full dataframe and among macro variables.

NOTE: can use VIF (e.g. via `cars::vif()`*) to further investigate multicollinearity between categorical/continuous predictors.*

Variable Selection

Given the context of the PD model and the situations where such a model is typically deployed (loan acceptance/rejection, loan pricing, expected loss calculations, etc.), we are more concerned with out-of-sample performance than raw statistical significance on the training data. For this reason, we use cross-validation (CV) for model selection instead of deviance tests on the entire dataset.

We still care about the final interpretation of model parameters, so we will also qualitatively consider the interpretability of each model when reviewing the results of CV.

We use rolling CV to compare alternative model specifications for loan age. Due to the panel structure of the data, we can't easily use a pre-made function for time series CV such as `timetk::time_series_cv()`, so we implement a custom approach[6]. Manually implementing cross-validation would be very computationally expensive given our large dataset (6.1M+ rows), so we:

1. make use of the 'data.table' package[7] to speed up dataframe operations, and
2. use a single relatively small sample of loans in all folds of CV.

In particular, we use 10-fold CV with 30 month training windows and 8 month validation windows.

Given the binary outcome of our data, we use binary entropy[8] as our loss function to compare model specifications.[9]

We proceed with a top-down approach for variable selection.

- Loan age is known to typically be a key risk factor, often with nonlinear association, so we use CV for selecting the functional form for the loan age term of the model as described above.
- Moreover, we will actually use Ridge GLM (L2 regularized) instead of the plain GLM model for these CV folds. This is because Ridge offers a solution to these two critical issues:
- Firstly, there will likely be some categorical predictors with only one unique level for some folds, in which case 'glm()' will fail to fit the model because it doesn't know how to assign a coefficient to the indicator variable for the factor level with no observations.
- Secondly, due to the outcome (default) being a rare event, we can have *separation*; there may be a level of a factor for which the data only has one type of event (only defaults or no defaults at all), which causes unstable estimate from ordinary Maximum Likelihood Estimation (MLE) because the optimization routine will iteratively push the estimate towards $\pm\infty$.
- After choosing the specification for laon age, we will consider if we can drop any other variables. For the other variables, we have a fairly large collection of continuous and discrete predictors, so we use Lasso (L1 regularization) to decide what to include in the model.
- After CV and Lasso, we will fit the un-regularized model with the chosen predictors. If there are statistically insignificant predictors in the last model, we will re-assess multicollinearity

[6] Even in Python, we'd have to resort to a similar manual approach.

[7] The 'data.table' package saves a lot of computational cost in large part by avoiding the creation of as many deep copies of dataframes as done in base R or 'dplyr'.

[8] a.k.a. the negative log-likelihood (NLL) for an IID Bernoulli sample, a.k.a. the log loss (as called in 'sklearn' in Python).

[9] Isn't this flawed because that loss function assumes data is IID (we have panel data so the same loans are auto-correlated across time)? CHECK THIS

and use CV to compare a reduced model to the full model.

Lastly, note that we will do all variable selection work on a training subset (chosen to be the first 80% of time points) while setting aside a testing subset (last 20% of time points) for final model calibration. These global subsets will be created before any CV is conducted. The train/test folds in CV for loan age specification will be subsets of the global training set.

Set up Global Train/Test Sets

```
# Load libraries for custom CV implementation
library(data.table)
library(splines)
library(glmnet) # for fitting regularized GLMs

# Create global training and holdout subsets
train_prop <- 0.8
global_train_end <- ceiling(length(months) * train_prop)
train_mos <- months[1:global_train_end]
df_train <- full_df %>% filter(date %in% train_mos)
df_test <- full_df %>% filter(date %notin% train_mos)
```

The `glmnet` package has functions for fitting an Elastic Net GLM, but this model can be reduced into a Lasso or Ridge GLM by specifying the `alpha` parameter to be `0` or `1`.

CV for Age of Loan Specification

```
# Prepare data.table for CV
dt <- as.data.table(df_train)
dt[, date := as.IDate(date)]
dt[, y := as.integer(y)]

# Encode categorical variables in data.table as factors
cat_vars <- c("deliq_num", "occupancy_status", "region",
              "loan_purpose")
for (v in cat_vars) dt[, (v) := as.factor((v))]


# Create random sample of loans to be used for CV
set.seed(1) # reproducibility
sample_frac <- 0.15
all_ids <- unique(dt$loan_sequence_number)
sample_ids <- sample(all_ids, size=ceiling(sample_frac*length(all_ids)), replace=FALSE)
dts <- dt[loan_sequence_number %in% sample_ids] # sampled data

cat("Sampled", length(sample_ids), "loans out of", length(all_ids), "\n")
cat("Sampled", nrow(dts), "rows out of", nrow(dt), "\n")

dts <- dt
```

```r
# Create table telling us which rows correspond to each time period
date_tab <- dts[, .N, by=date][order(date)]
date_tab[, end_row := cumsum(N)]
date_tab[, start_row := shift(end_row, fill = 0) + 1]


# Function to map data into the indices defining each CV fold
make_folds <- function(date_tab, init_periods, assess_periods, step) {

  m <- nrow(date_tab)
  stopifnot(init_periods + assess_periods <= m)

  fold_splits <- seq(from=init_periods, to=m-assess_periods, by=step)

  return(
    data.table(
      fold = seq_along(fold_splits),
      train_end_row  = date_tab$end_row[fold_splits],
      test_start_row = date_tab$start_row[fold_splits + 1],
      test_end_row   = date_tab$end_row[fold_splits + assess_periods],
      train_end_date = date_tab$date[fold_splits]
      )
  )
}

# Create CV folds
folds <- make_folds(date_tab, init_periods=30, assess_periods=8, step=8)
cat("Number of folds:", nrow(folds))



# Vector of all non-age covariate names
other_x <- setdiff(names(dts),
                   c("loan_sequence_number", "date", "y", "loan_age"))

# Right-hand-side formula string
rhs <- paste(other_x, collapse=" + ")

# Create list of different model formulas
specs <- list(
  age_linear = as.formula(paste0("y ~ loan_age_pct + ", rhs)),
  age_log1p = as.formula(paste0("y ~ log1p(loan_age_pct) + ", rhs)),
  bins = as.formula(paste0("y ~ age_group_factor", rhs)),
  age_spline = as.formula(paste0("y ~ ns(loan_age_pct, df=4) + ", rhs))
)
```

```r
# Binary entropy loss function (with padding for numerical stability)
be_loss <- function(y, p, eps=1e-15){
  p <- pmin(pmax(p, eps), 1-eps)
  return(-mean(y*log(p) + (1-y)*log(1-p)))
}


# Function to evaluate a model specification on CV folds
eval_spec <- function(x, y, folds, loss_func) {

  n_folds <- nrow(folds)
  out_list <- vector("list", n_folds)  # store per-fold results

  for (i in seq_len(n_folds)) {

    tr_end <- folds$train_end_row[i]
    te_s <- folds$test_start_row[i]
    te_e <- folds$test_end_row[i]

    x_train <- x[1:tr_end, , drop = FALSE]
    x_test <- x[te_s:te_e, , drop = FALSE]
    y_train <- y[1:tr_end]
    y_test <- y[te_s:te_e]

    # Fit Lasso logistic regression
    fit <- glmnet(
      x = x_train,
      y = y_train,
      family = "binomial",
      alpha = 1, # alpha = 1 for Lasso
      nlambda = 50
    )

    # Choose the last lambda for prediction
    lambda_use <- tail(fit$lambda, 1)

    p <- as.numeric(
      predict(fit, newx = x_test, type = "response", s = lambda_use)
    )

    loss <- loss_func(y_test, p)

    # one-row data.table for this fold
    out_list[[i]] <- data.table(
      fold = folds$fold[i],
      loss = loss,
      train_end_month = folds$train_end_month[i]
```

```
    )

    cat("\t", "Done fold ", i) # status update
  }

  # combine all folds
  rbindlist(out_list)
}


# Conduct CV (train and test the models on each fold)
cv_results <- rbindlist(
  lapply(names(specs), function(model) {

    cat("Working on model", model) # status update

    formula <- specs[[model]]

    needed_vars <- all.vars(formula)
    dts_small <- dts[, ..needed_vars]

    x_mat <- model.matrix(formula, data = dts_small)[, -1, drop = FALSE]
    y_vec <- dts_small$y

    res <- eval_spec(x_mat, y_vec, folds, be_loss)
    res[, spec := model]
    res
  })
)


cv_summary <- cv_results[, .(
  mean_loss = mean(loss),
  sd_loss = sd(loss),
  se_loss = sd(loss)/sqrt(.N)
  ),
  by = spec][order(mean_loss)]

cv_summary


best_spec <- cv_summary$spec[1]
best_formula <- specs[[best_spec]]
cat("Best specification for loan age component of model:", best_spec)
```

Decide on final spec to use while taking into account interpretability

STILL NEED TO REBOOT R THEN DO THE CV ABOVE (IT WILL TAKE A WHILE)

Lasso for Large-Scale Variable Selection

We use the ability of Lasso to shrink coefficient estimates to zero to help determine what variables to keep. Since factors will be converted into a collection of indicators for each level, if Lasso ends up dropping some but not all of these indicators then we will still keep that factor in the model.

Interactions

NOTE TO SELF: suggest a few economically plausible interactions and test them via CV. - examples: loan age × delinquency history LTV × house price index DTI × income band origination vintage × macro covariate credit score × utilization loan purpose × occupancy status

```
# Fit model 1: main effects
#m1 <- glm(..., family=binomial, data=full_df)
```

Train and Evaluate Final Model

We now test the model selected from earlier on "out-of-time" (test set) data.

Interpretation

(core parameter interpretation)

Prediction Under Stress Testing Scenarios

(time permitting: predict over made up stress scenarios like high inflation + low-mid unmeployment)

Limitations and Possible Improvements

- Freddie Mac Sampling Bias
- Definition of Default Event
  - can loosen def to not force default to be absorbing state (dont drop rows after they default)
    - could use count or indicator of previous defaults as a covariate in the model
- Variables to Investigate:
  - macroeconomic uncertainty (e.g. JLNUM3M)
  - policy uncertainty (e.g. USEPUINDXD)
  - try creating an approximate dynamic DTI variable (after origination)
    - use income and/or debt indices similar to how HPI was used to estimate dynamic LTV
    - this would be more challenging than estimating dynamic LTV since:
      - both debt and income are dynamic whereas only home value was dynamic for LTV
      - income distribution has more skew and/or kurtosis than home prices (right?)
  - try using the un-normalized current actual unpaid principal balance variable (`current_actual_upb`)
- Dealing with Class Imbalance

  - try oversampling or importance sampling
- Alternative Models to Compare
  - Poisson regression for the *count* of defaults
  - XGBoost or other newer classification models
  - Neural network using SHAP values for interpretation